POC-01: Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

Beginner Implementation Guide

Manual Azure setup, Python upload, troubleshooting, verification and Terraform recreation

How to use this guide

This handbook rebuilds the completed POC from scratch in the same order a beginner should follow. The manual Azure implementation is the baseline. Python is used to land test data securely. Terraform is presented afterward as a separate infrastructure-as-code recreation path.

Important: two implementation variants exist

The completed manual pipeline uses a 5-row CSV with an order_date column and a simple MERGE/log design. The repository also contains an advanced 30-row lab using order_ts, rejects, file-level control and watermark logic. Do not mix the two schemas accidentally.

Recommended learning sequence

- Build the Azure resources manually and understand what each service does.
- Create and test ADF linked services, datasets and the pipeline.
- Run the 5-row manual CSV end to end and verify SQL, archive and monitoring.
- Run the Python uploader using Microsoft Entra authentication.
- Perform idempotency, missing-file and RBAC negative tests.
- Recreate the infrastructure with Terraform using separate names or after cleanup.
- Optionally migrate to the advanced 30-row validation/watermark variant.

Contents - Part 1

Contents - Part 2

1. Objective and business scenario

A retail company receives daily order CSV files. The platform must land raw input safely, orchestrate processing, load relational staging, merge business rows without duplicates, preserve processed source files for audit and expose execution evidence.

Business requirements

- Accept a new file in ADLS landing.
- Check that the expected file exists before processing.
- Load into SQL staging, then merge on the business key.
- Re-running the same input must not duplicate curated orders.
- Archive the raw file only after successful processing.
- Log execution status and verify activity results independently.

Target data flow

Architecture flow

```
Local CSV / Python uploader
        |
        v
ADLS Gen2: landing/orders/YYYY/MM/DD/orders_001.csv
        |
        v
ADF: PL_INGEST_ORDERS_BATCH
  1. Get_Metadata_File
  2. If_File_Exists
  3. Script_Log_Start
  4. Copy_Orders_To_Staging
  5. SP_Merge_Orders
  6. Copy_Archive_File
  7. Delete_Landing_File
  8. Script_Log_Success
        |
        +--> archive/orders/YYYY/MM/DD/...
        v
Azure SQL: dbo.orders_stg -> dbo.orders
        |
        v
ADF Monitor + SQL verification
```

Azure concepts used in this POC

Resource Group	Lifecycle boundary for the POC. Deleting it is the simplest cost-control cleanup.
ADLS Gen2	StorageV2 account with hierarchical namespace. Holds landing, archive and quarantine zones.
Azure Data Factory	Orchestrates file checks, copy, SQL procedure execution, archive and delete operations.
Azure SQL Database	Contains staging, curated orders and ETL logging/control objects.
Managed Identity	Lets ADF request Microsoft Entra tokens without storing an application password.
Azure RBAC	Grants the ADF identity and developer identity only the required ADLS data access.
Azure Monitor / ADF Monitor	Provides run status, activity duration, row counts and failure details.
Terraform	Recreates the Azure foundation as code after the manual learning path is understood.

2. Prerequisites, names and cost guardrails

Windows prerequisites

- Python 3.12 is fine for this POC.
- Poetry environment with azure-identity and azure-storage-file-datalake.
- Azure CLI is a separate Windows application and is required for the az command.
- Git and VS Code are recommended.
- Terraform 1.6+ is required only for the IaC phase.

Neutral naming pattern

Resource Group	rg-azde-poc01-dev
Storage Account	stazdepoc01
Data Factory	adf-azde-poc01-
SQL logical server	sql-azde-poc01-
SQL Database	sqldb-azde-poc01-dev
Key Vault	kv-azde-poc01-

Cost rules

- Use a tiny development Azure SQL configuration available in your region.
- Run ADF only on demand.
- Keep test files tiny.
- Treat Log Analytics and Private Endpoints as optional learning additions.
- Delete the POC Resource Group after evidence capture.

3. Manual setup - Create the Resource Group

- Sign in to Azure Portal and confirm the correct tenant and subscription.
- Open Resource groups and choose Create.
- Create rg-azde-poc01-dev in a region supported by all required services.
- Add project, environment, owner and autoDelete tags.
- Optionally create a small Cost Management budget with email thresholds.

Beginner tip

A Resource Group is not a folder only. It is also a lifecycle and access-management boundary. Keeping this POC isolated makes cleanup safer.

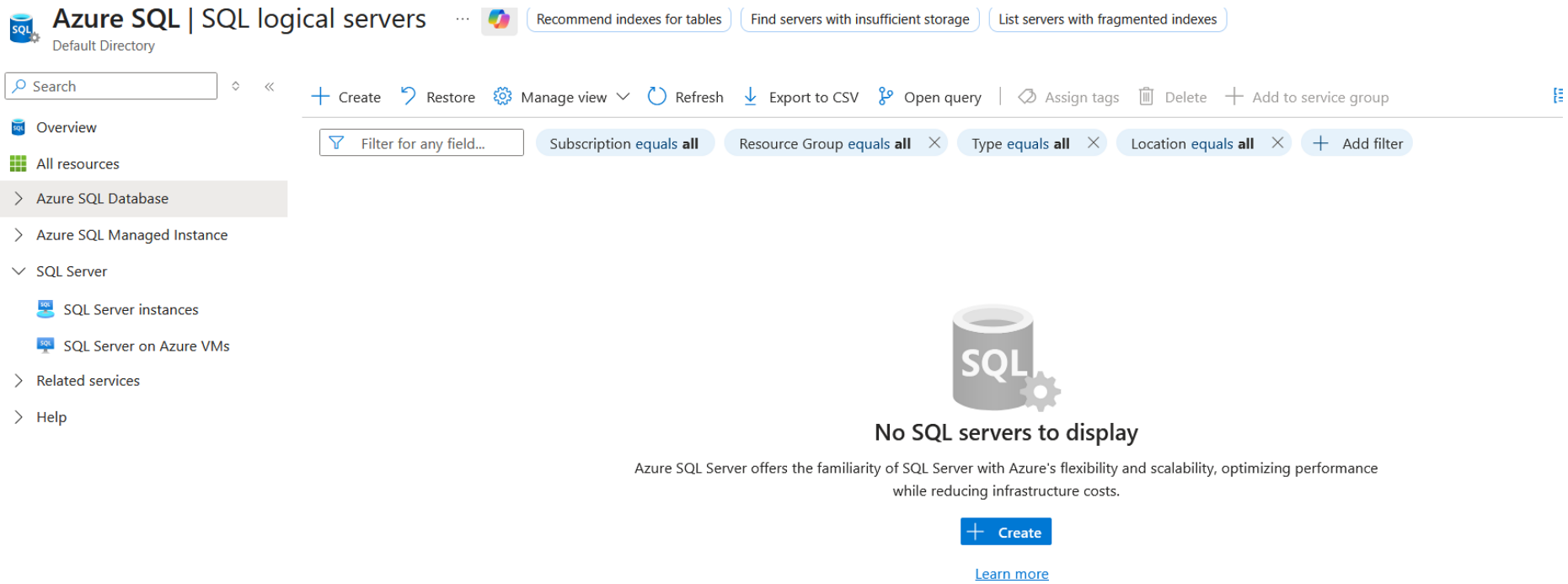
Manual setup - Create ADLS Gen2

- Create a StorageV2 account in the POC Resource Group.
- Use Standard performance and LRS for the small lab.
- Enable Hierarchical namespace.
- Keep anonymous/public blob access disabled.
- Create private containers named landing, archive and quarantine.
- Use landing/orders/2026/08/28/orders_001.csv as the manual sample path.

Why three containers?

landing is the active ingestion zone; archive preserves successfully processed raw files; quarantine holds invalid or failed evidence.

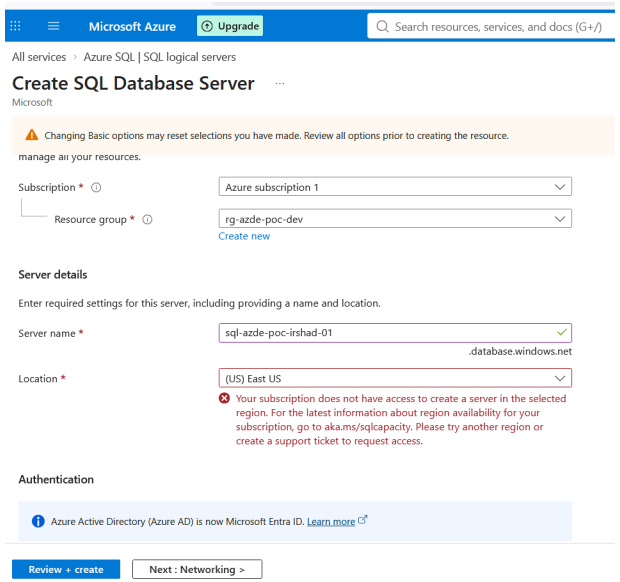
Manual Azure setup - Azure SQL service



Open the Azure SQL service and begin creation of the logical server/database.

What this proves: This is the starting point for the relational side of the POC.

Issue - SQL region/capacity restriction



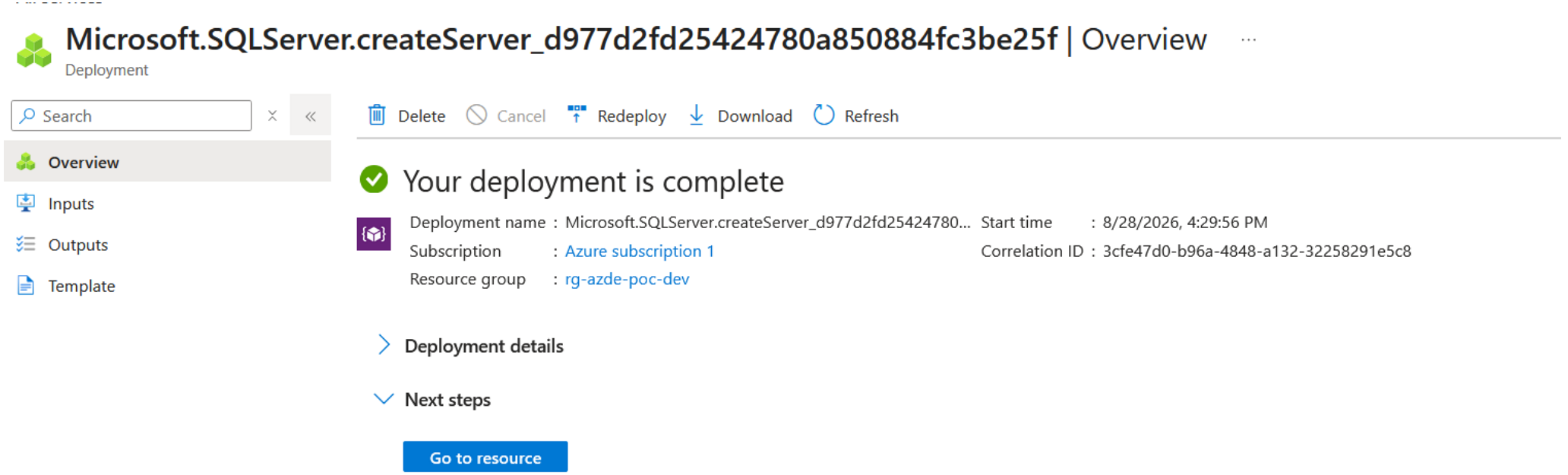
A region/capacity restriction was encountered while creating Azure SQL.

What this proves: Azure SQL creation may be limited by subscription or regional capacity.

Issue and fix

Choose another region supported by the subscription and the selected SQL configuration, then create the server/database there. Keep related resources in the same region where practical.

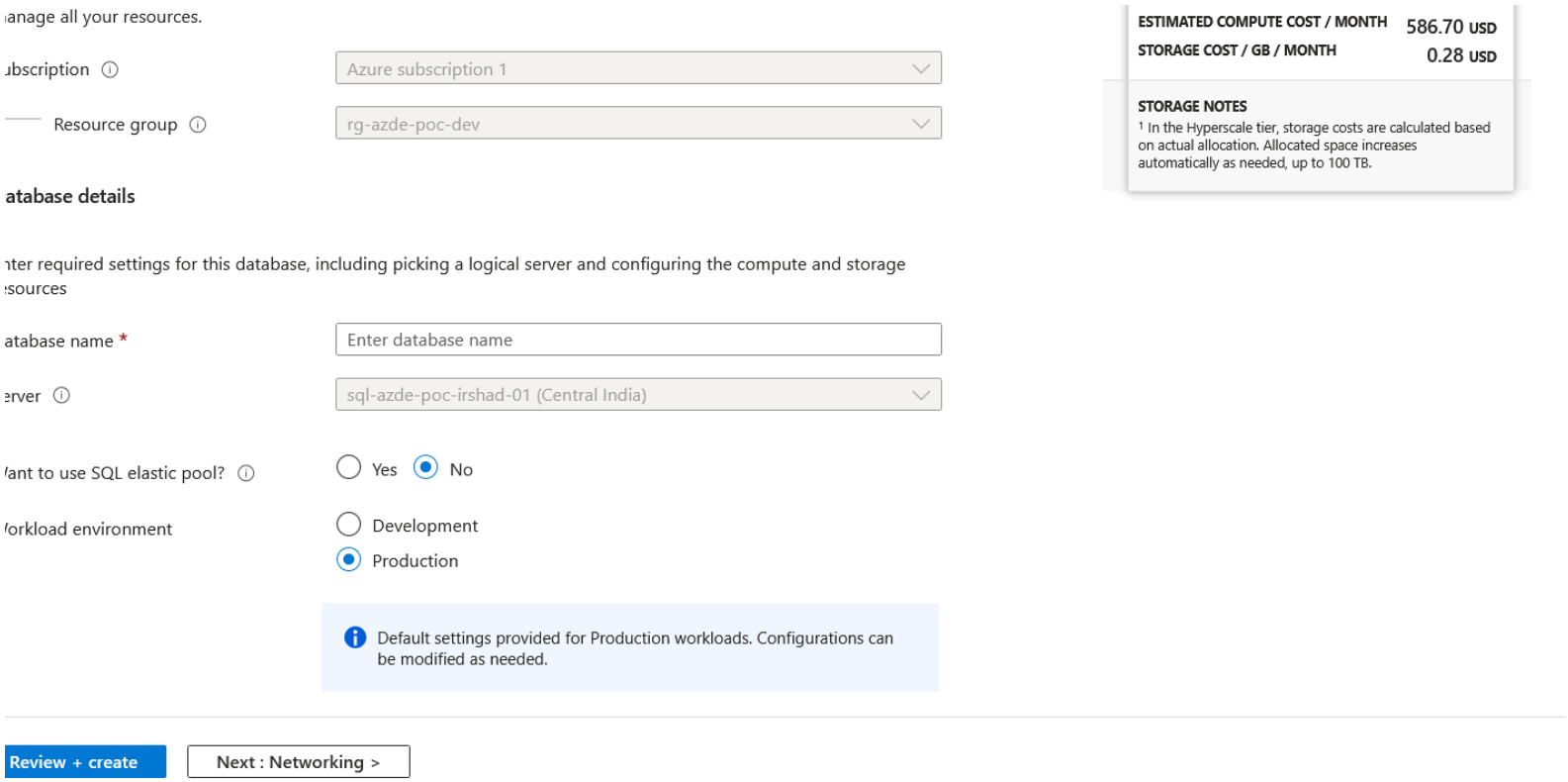
Manual Azure setup - SQL logical server created



The Azure SQL logical server deployment completed successfully.

What this proves: The server resource now exists and can host the POC database.

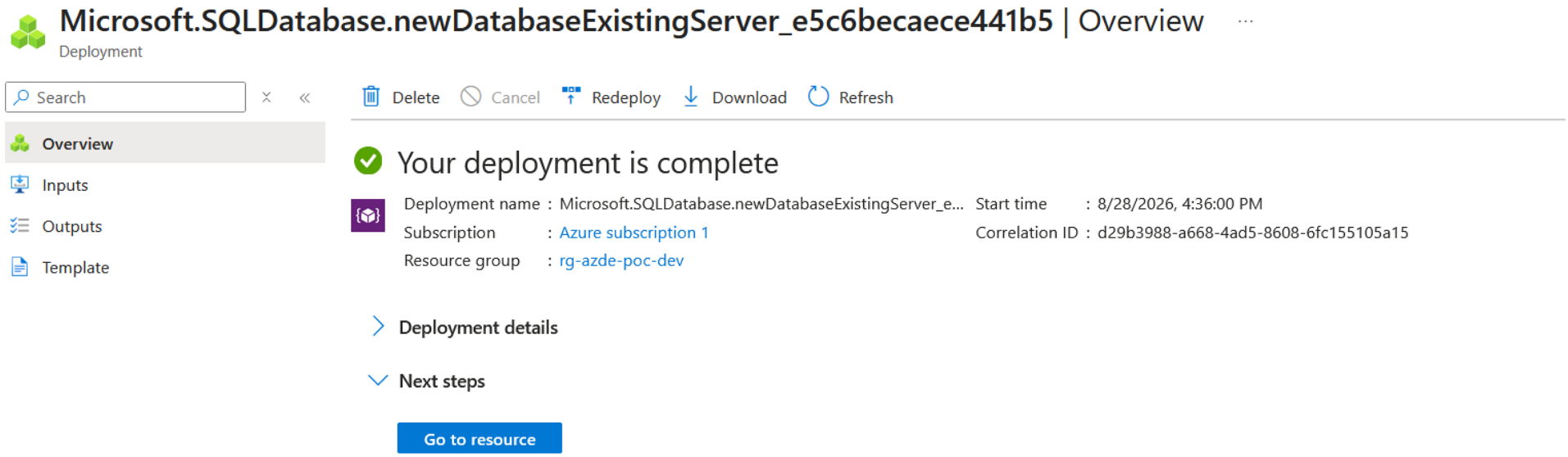
Manual Azure setup - Configure SQL Database



Configure the SQL Database under the selected logical server.

What this proves: Use the small lab database name and a development SKU available in the chosen region.

Manual Azure setup - SQL Database created



The POC Azure SQL Database deployment completed successfully.

What this proves: The target database is ready for firewall, Microsoft Entra and schema configuration.

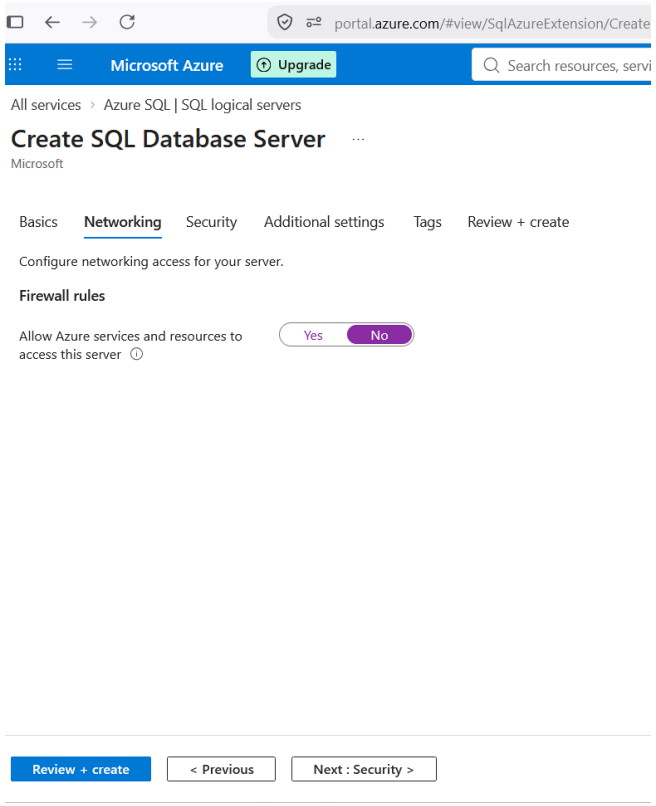
4. Manual setup - SQL networking and firewall

- Open the Azure SQL logical server, not only the database.
- Open Networking.
- For this beginner public-endpoint lab, enable Allow Azure services and resources to access this server.
- Add your current client IP when browser/query/client access from your laptop requires it.
- Save the networking changes.

Production note

This public-endpoint exception is a beginner-lab simplification. A hardened design uses private connectivity and still requires identity/database authorization.

SQL networking evidence



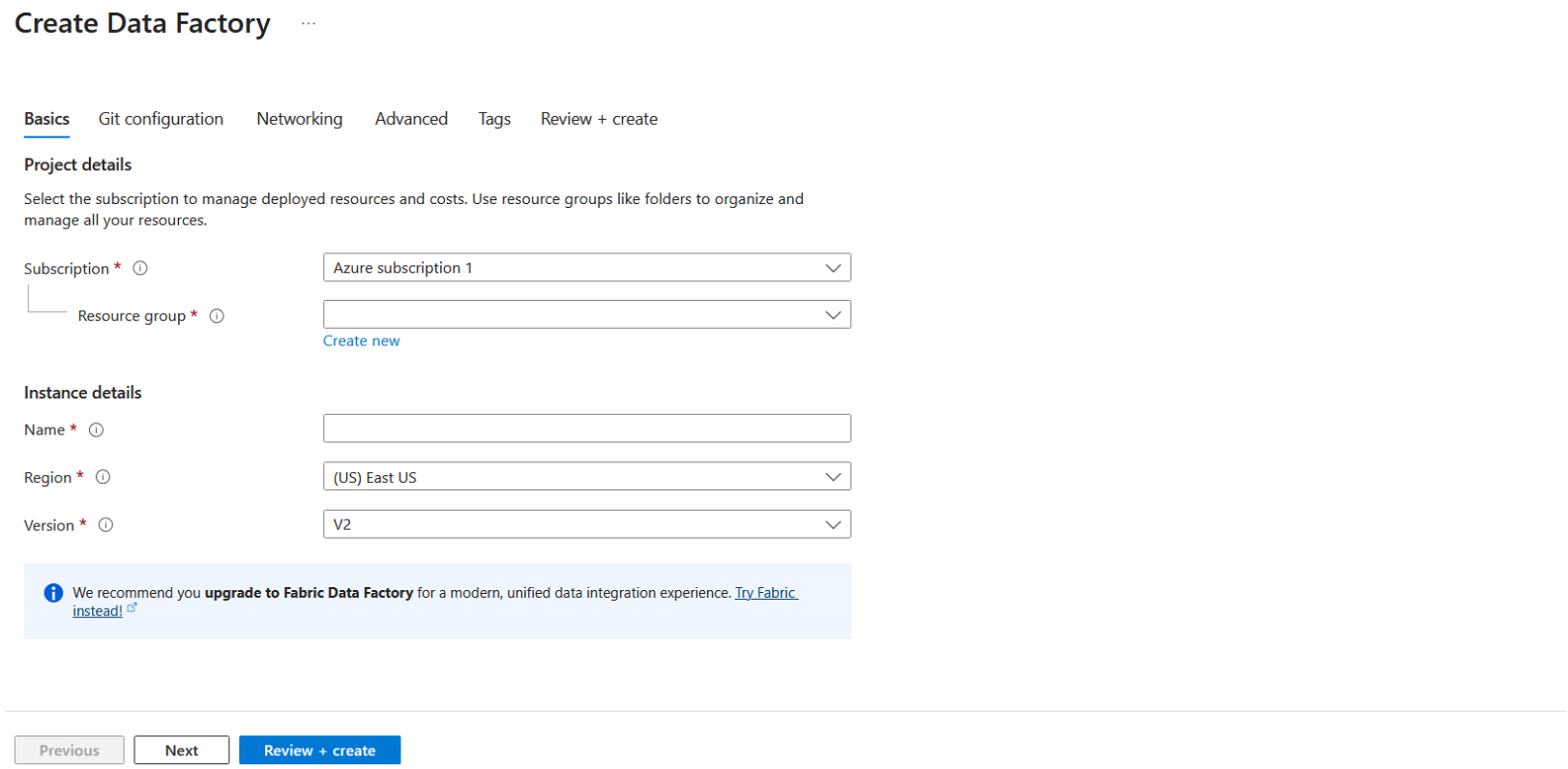
Azure SQL server Networking configuration showing the beginner-lab Azure services firewall option.

What this proves: This setting was required for the ADF Azure Integration Runtime to reach the SQL server in the manual lab.

5. Manual setup - Create Azure Data Factory

- Create a Data Factory in the POC Resource Group.
- Configure Git later for the first manual run.
- Open the deployed Data Factory and launch ADF Studio.
- Use the system-assigned Managed Identity for ADLS and SQL connectivity.

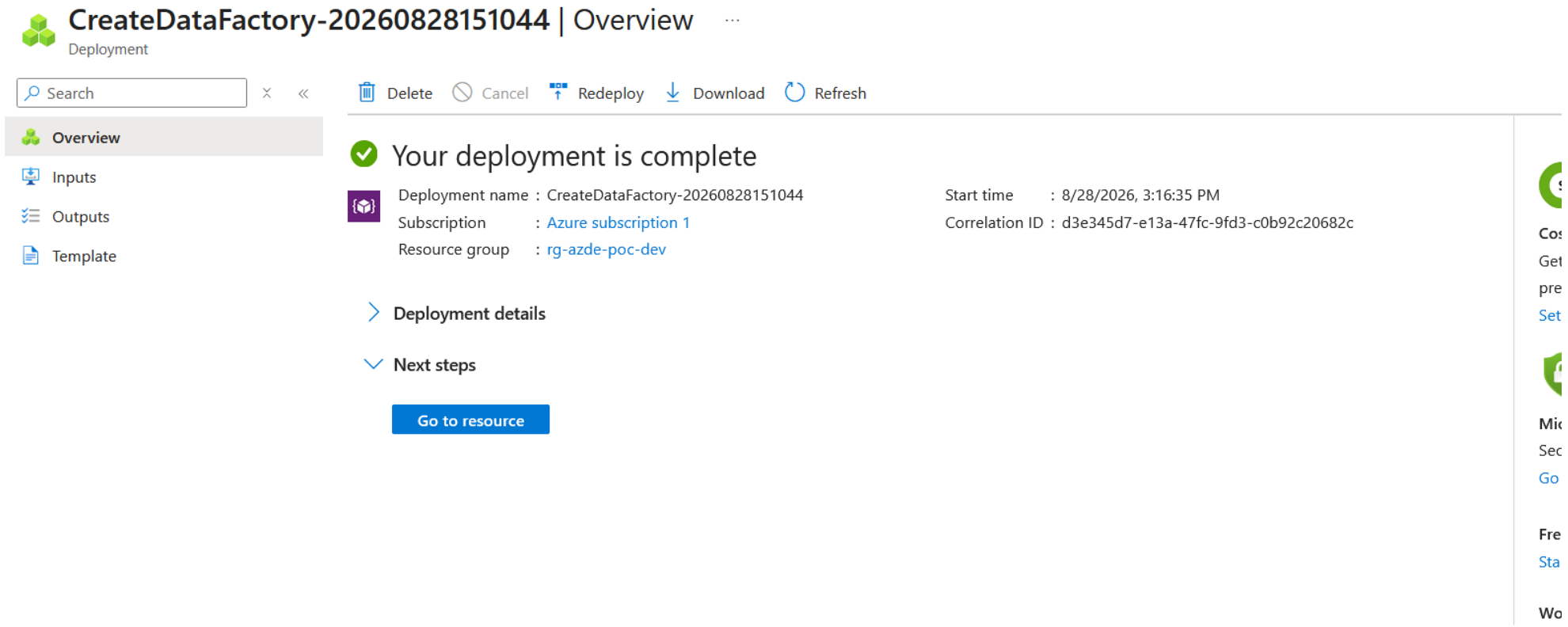
Create Azure Data Factory



Data Factory Basics configuration during manual creation.

What this proves: The factory is placed in the same POC lifecycle boundary.

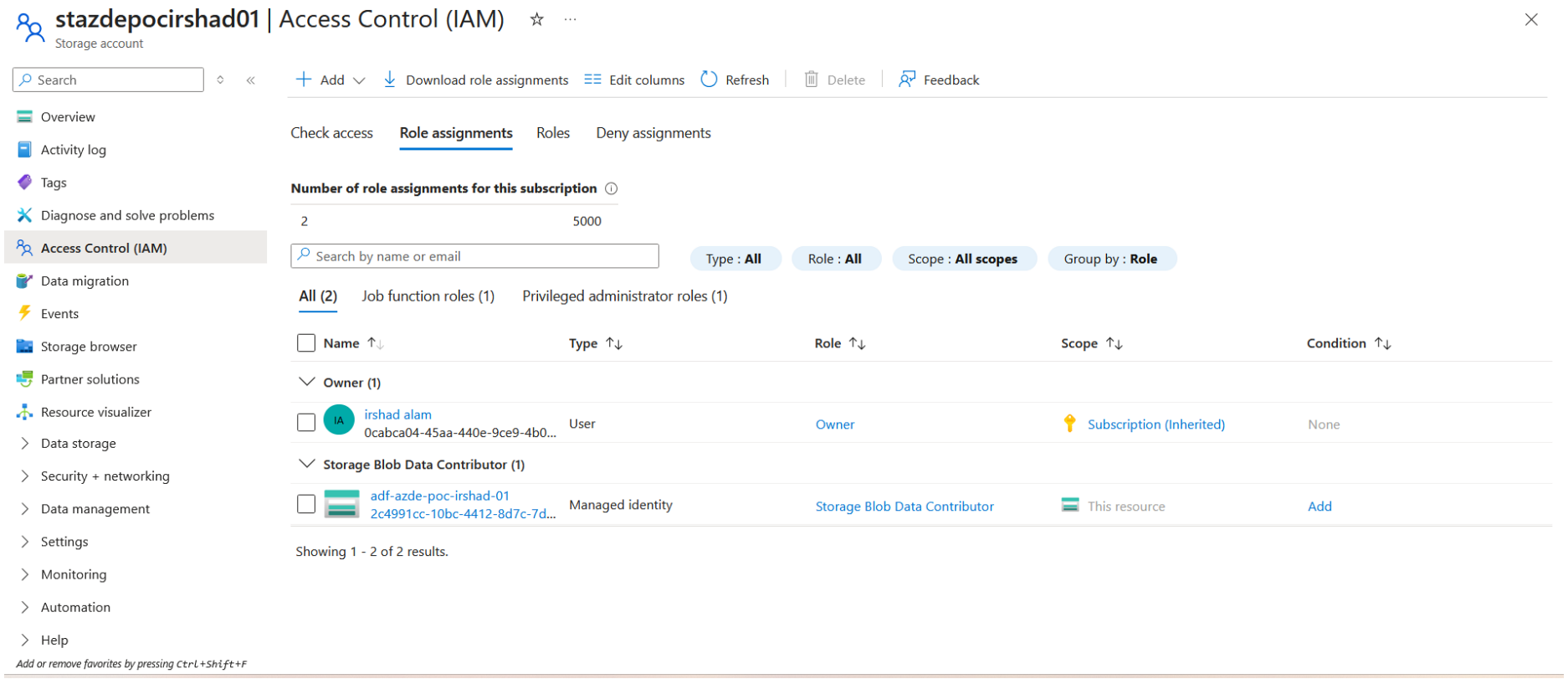
Azure Data Factory deployment completed



The Data Factory resource deployed successfully.

What this proves: The next steps are RBAC, SQL database principal creation and ADF authoring.

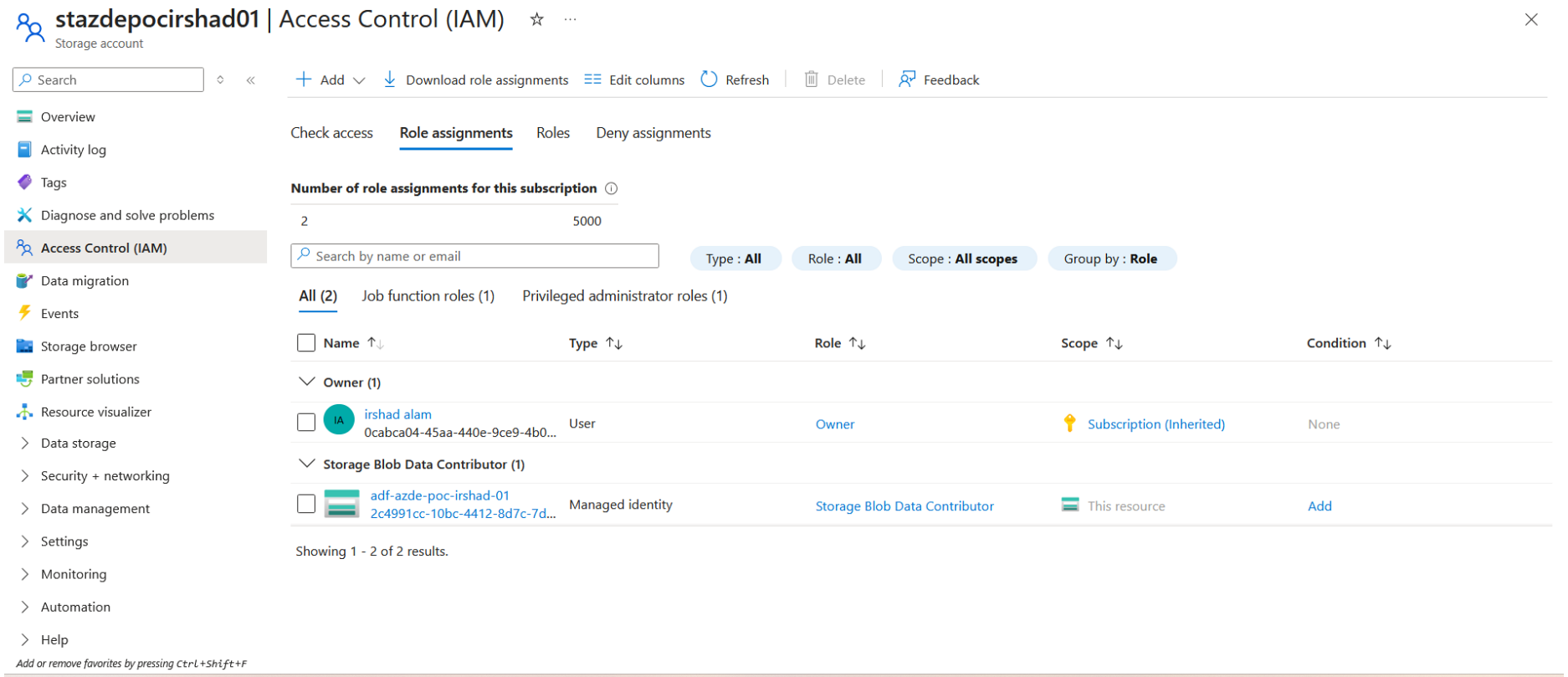
Grant Storage data-plane permission



Storage Account IAM role assignments for the developer and/or managed identity.

What this proves: The Python uploader and ADF are separate identities and need data-plane permission independently.

Verify Storage Blob Data Contributor



Storage IAM evidence for Storage Blob Data Contributor.

What this proves: ADF needs read, write and delete capability for landing/archive behavior.

Create the manual SQL objects

The completed manual pipeline uses an order_date schema. Run these objects in the POC database before authoring the ADF copy and stored procedure activities.

Manual DDL - logging and staging

```
IF OBJECT_ID('dbo.etl_file_log', 'U') IS NULL
BEGIN
  CREATE TABLE dbo.etl_file_log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    file_name VARCHAR(255) NOT NULL,
    file_size_bytes BIGINT,
    status VARCHAR(50) NOT NULL,
    rows_inserted INT DEFAULT 0,
    rows_updated INT DEFAULT 0,
    started_at DATETIME2,
    completed_at DATETIME2,
    error_message NVARCHAR(MAX)
  );
END;

IF OBJECT_ID('dbo.orders_stg', 'U') IS NULL
BEGIN
  CREATE TABLE dbo.orders_stg (
```

Manual DDL - logging and staging - Continued

```
    order_id INT, customer_id INT, order_date VARCHAR(50),
    product_id INT, quantity INT, unit_price DECIMAL(10,2),
    status VARCHAR(50)
  );
END;
```

Create the manual curated table

dbo.orders DDL

```
IF OBJECT_ID('dbo.orders', 'U') IS NULL
BEGIN
  CREATE TABLE dbo.orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    product_id INT,
    quantity INT,
    unit_price DECIMAL(10,2),
    total_amount AS (quantity * unit_price) PERSISTED,
    status VARCHAR(50),
    source_file VARCHAR(255),
    created_at DATETIME2 DEFAULT SYSUTCDATETIME(),
    updated_at DATETIME2 DEFAULT SYSUTCDATETIME()
  );
END;
```

Create the manual MERGE stored procedure

```
CREATE OR ALTER PROCEDURE dbo.usp_merge_orders
  @p_file_name VARCHAR(255)
AS
BEGIN
  SET NOCOUNT ON;

  MERGE dbo.orders AS tgt
  USING (
    SELECT order_id, customer_id, TRY_CONVERT(DATE, order_date) AS order_date,
           product_id, quantity, unit_price, status
    FROM dbo.orders_stg
  ) AS src
  ON tgt.order_id = src.order_id
  WHEN MATCHED THEN UPDATE SET
    tgt.customer_id = src.customer_id, tgt.order_date = src.order_date,
    tgt.product_id = src.product_id, tgt.quantity = src.quantity,
    tgt.unit_price = src.unit_price, tgt.status = src.status,
    tgt.source_file = @p_file_name, tgt.updated_at = SYSUTCDATETIME()
  WHEN NOT MATCHED THEN INSERT
    (order_id, customer_id, order_date, product_id, quantity, unit_price,
```

Create the manual MERGE stored procedure - Continued

```
      status, source_file, created_at, updated_at)
  VALUES
    (src.order_id, src.customer_id, src.order_date, src.product_id, src.quantity,
     src.unit_price, src.status, @p_file_name, SYSUTCDATETIME(), SYSUTCDATETIME());
END;
```

Create the ADF Managed Identity database user

Set a Microsoft Entra administrator on the Azure SQL logical server. Connect to the POC database as that administrator, then create a contained user whose name exactly matches the Data Factory resource.

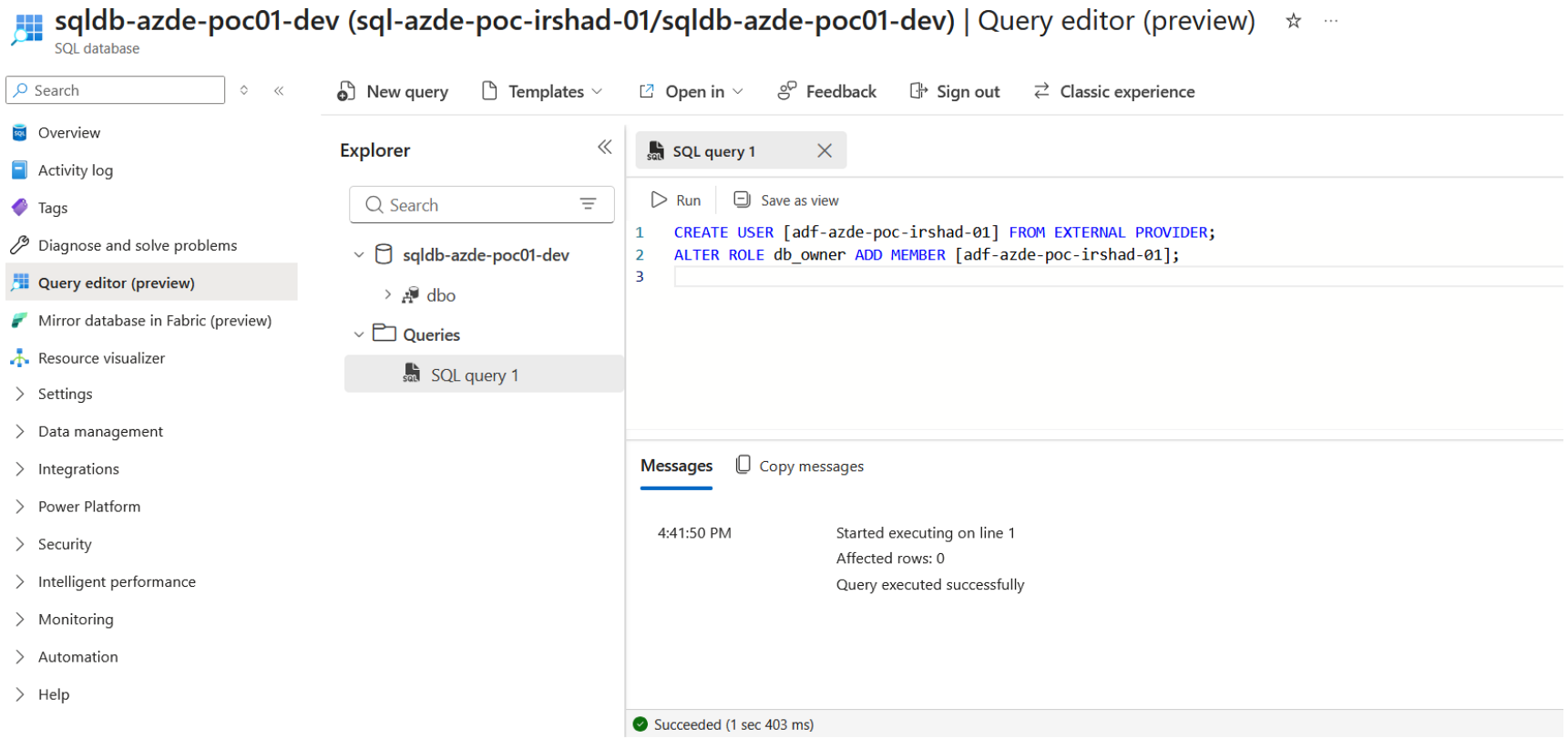
ADF database principal

```
CREATE USER [adf-azde-poc01-<unique>] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [adf-azde-poc01-<unique>];
ALTER ROLE db_datawriter ADD MEMBER [adf-azde-poc01-<unique>];
ALTER ROLE db_ddladmin ADD MEMBER [adf-azde-poc01-<unique>];
```

Least-privilege improvement

The manual lab used broad database roles for simplicity. A production design should grant only the exact table/procedure permissions required.

SQL permission evidence



Azure SQL Query Editor used to create the ADF Microsoft Entra database user and grant permissions.

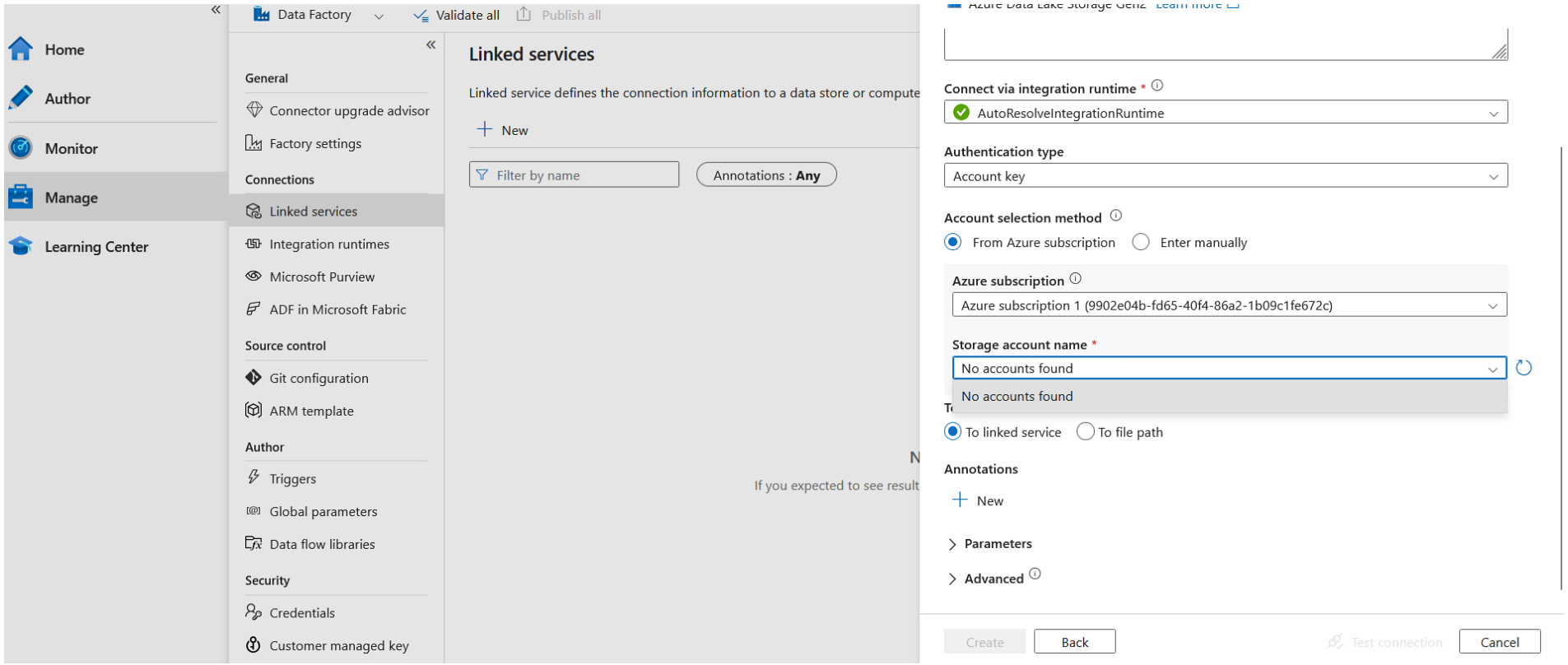
What this proves: This allows the ADF Managed Identity to authenticate to the database without a stored SQL password.

6. ADF linked services

LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2. Authentication: system-assigned Managed Identity. Target: POC Storage Account.
LS_AZURE_SQL_MI	Azure SQL Database. Authentication: system-assigned Managed Identity. Target: POC logical server and database.

- Test each linked service before creating datasets.
- For ADLS 403 errors, verify Storage Blob Data Contributor on the ADF identity.
- For SQL failures, verify SQL firewall, Microsoft Entra admin and the contained ADF database user.

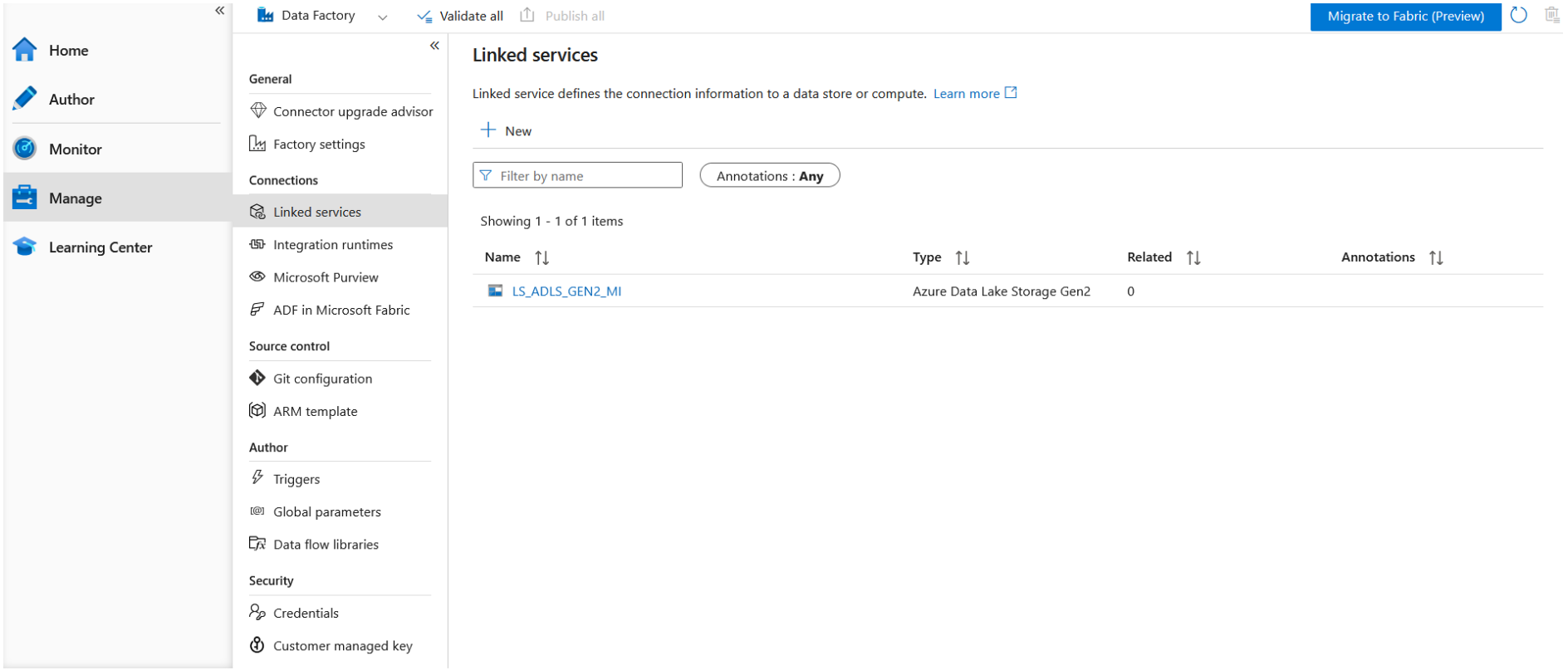
ADF linked service - ADLS Gen2



Create the ADLS Gen2 linked service with system-assigned Managed Identity.

What this proves: No Storage account key is required in ADF.

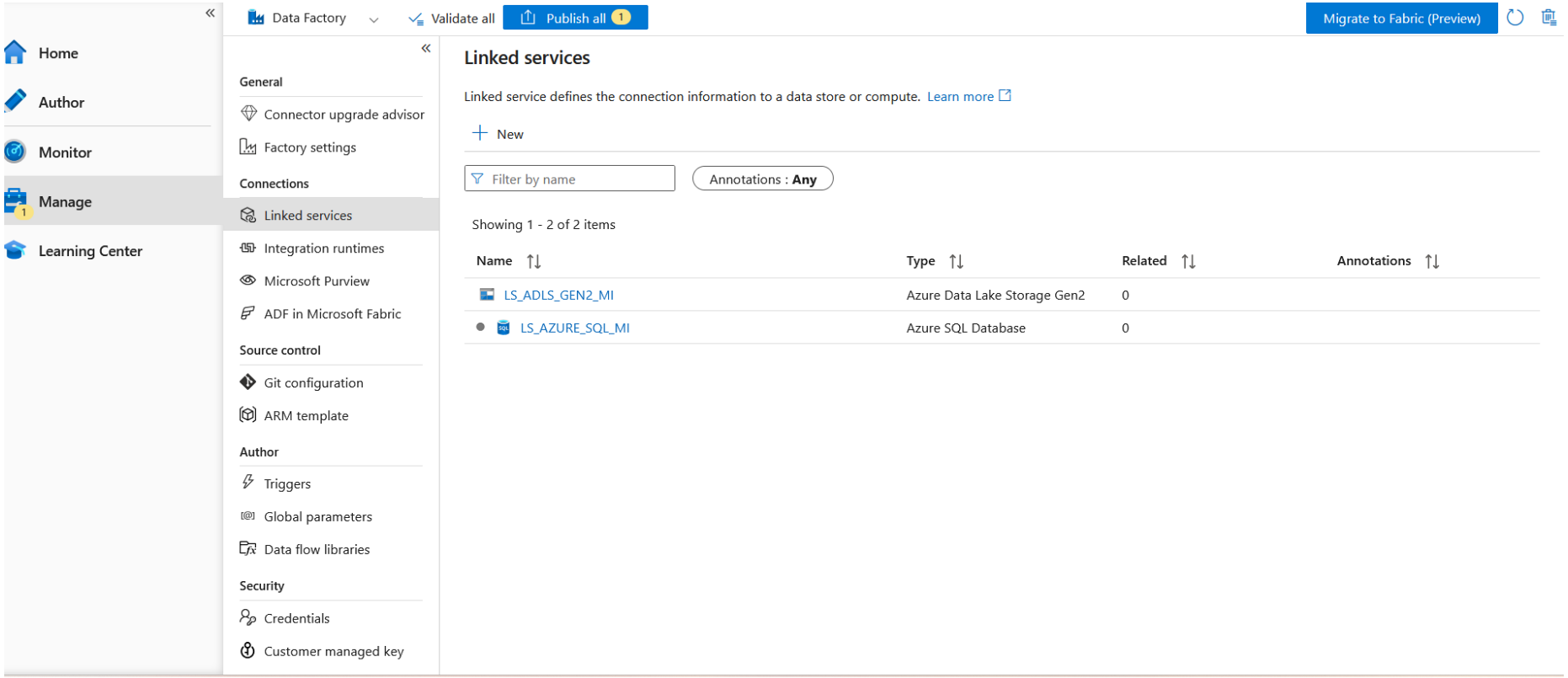
ADF linked service list - ADLS



ADF Manage view showing the ADLS linked service.

What this proves: Test connection should succeed before dataset authoring.

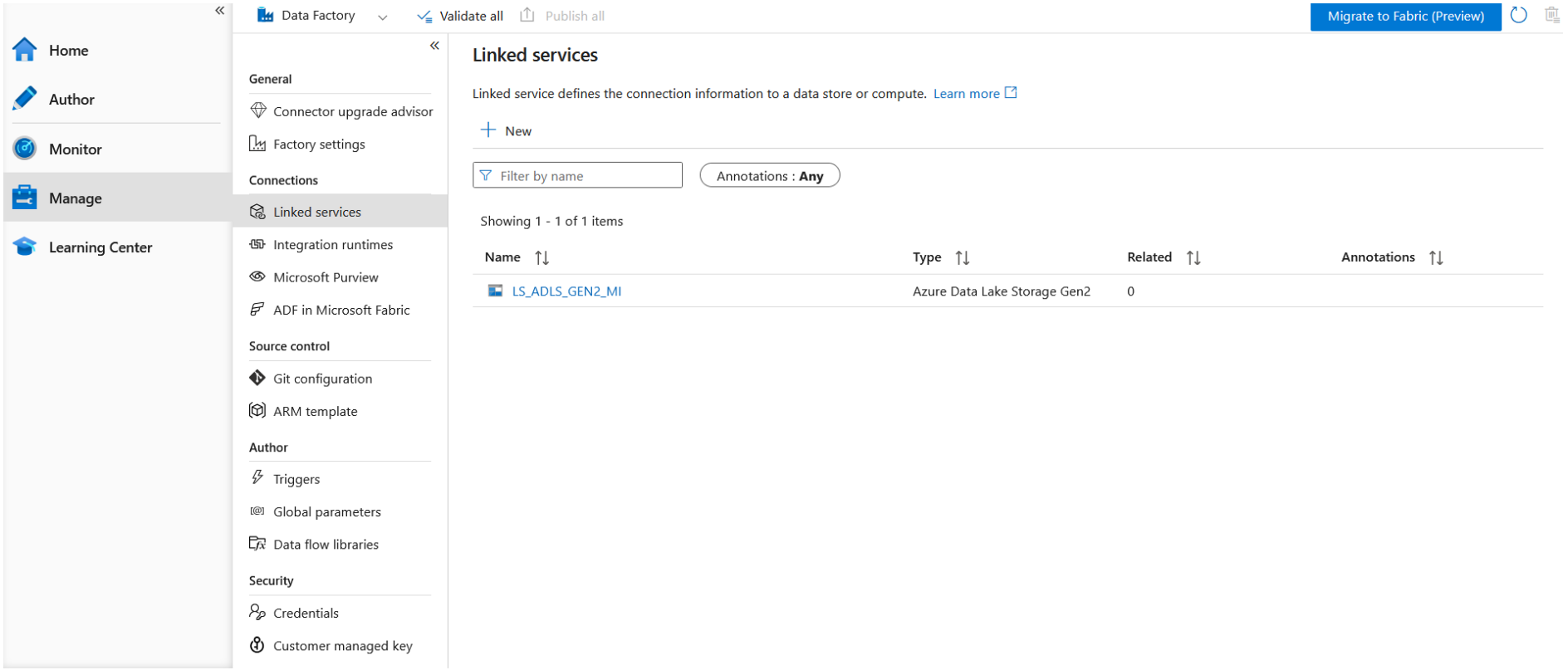
ADF linked services - ADLS and SQL



Linked-services list after adding Azure SQL connectivity.

What this proves: Both data stores are now available to datasets and activities.

ADF linked service - Azure SQL



Azure SQL linked-service configuration using Managed Identity.

What this proves: ADF authenticates as its Microsoft Entra identity rather than a SQL password.

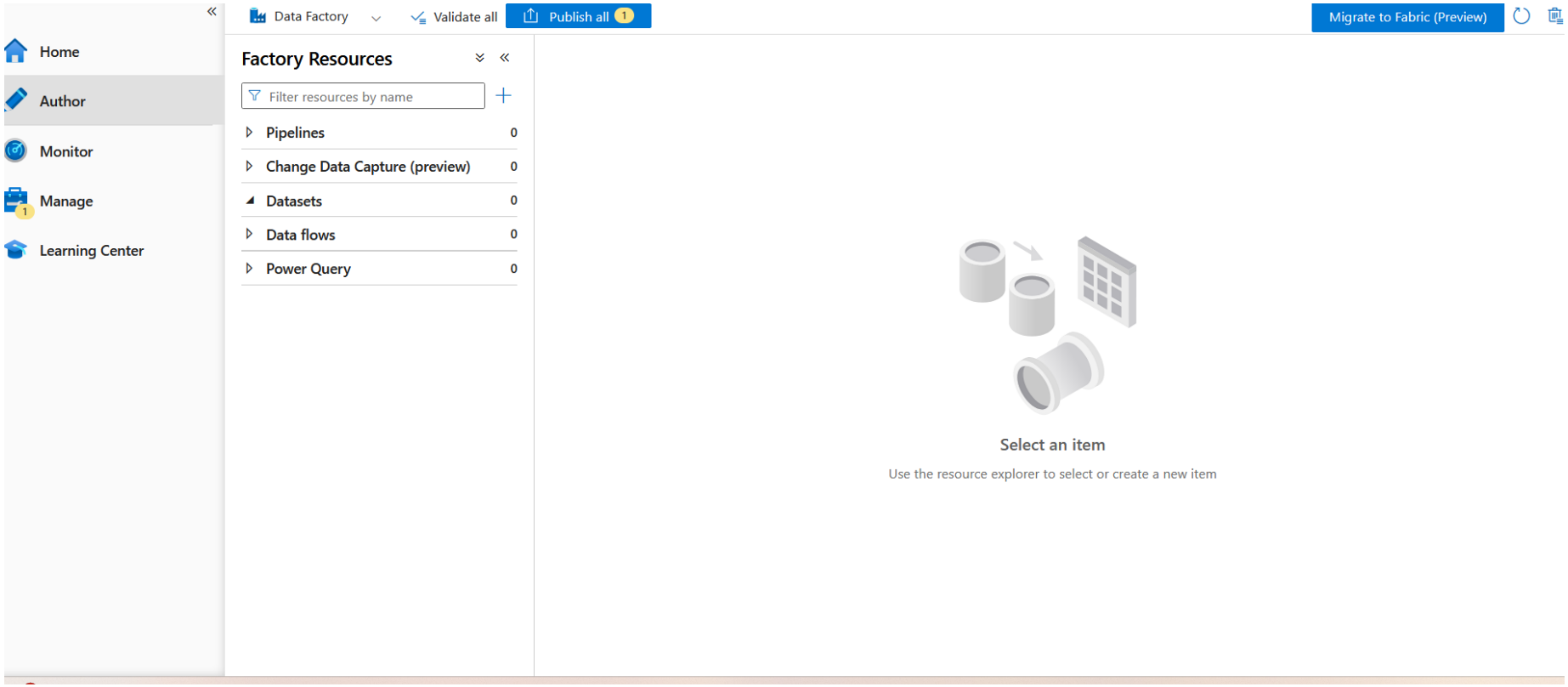
7. Create parameterized datasets

DS_ADLS_OrdersCsv	DelimitedText source. Parameters: p_container, p_folder, p_file. First row as header.
DS_ADLS_Binary	Binary dataset for byte-for-byte archive and Delete Activity. Same three path parameters.
DS_SQL_Table	Azure SQL Database dataset. Parameter: p_target_table. Schema dbo, table from parameter.

Dynamic dataset expressions

```
ADLS dynamic location
File system = @dataset().p_container
Directory   = @dataset().p_folder
File        = @dataset().p_file

SQL dynamic table
Schema = dbo
Table  = @dataset().p_target_table
```

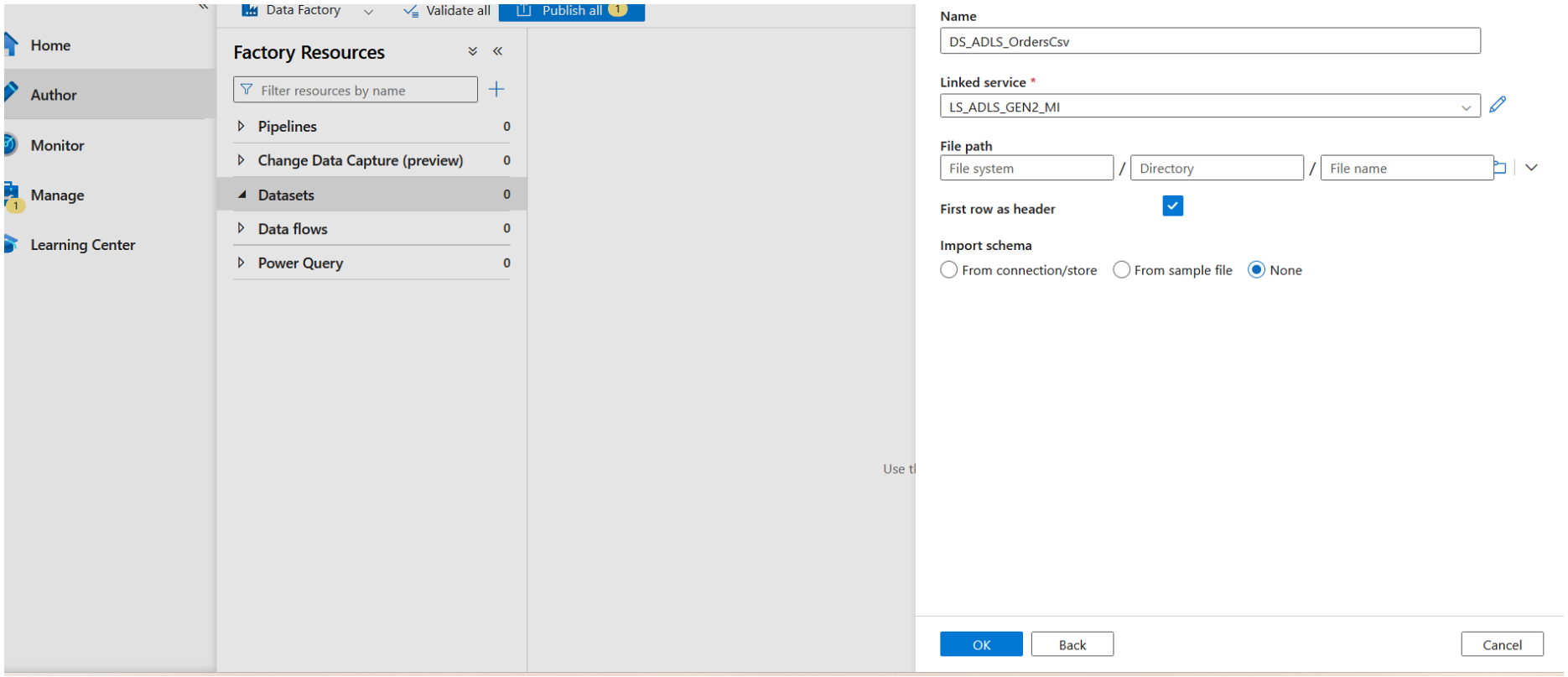
ADF authoring - begin datasets and pipeline



ADF Studio Author view at the start of dataset/pipeline authoring.

What this proves: The factory has the required authoring workspace and resources.

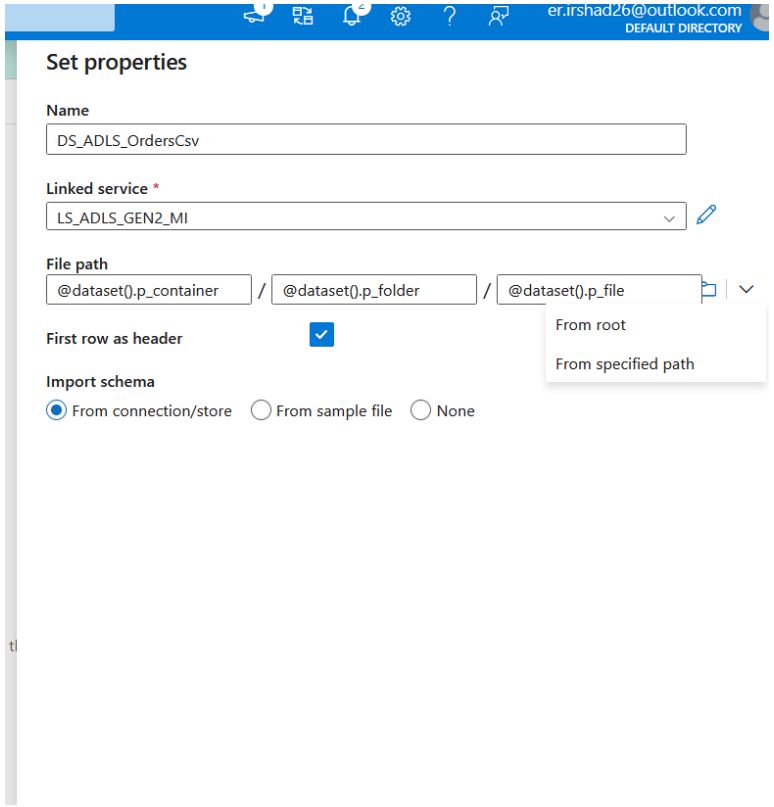
ADF dataset creation



Dataset creation properties during manual authoring.

What this proves: Create the source and sink datasets before wiring pipeline activities.

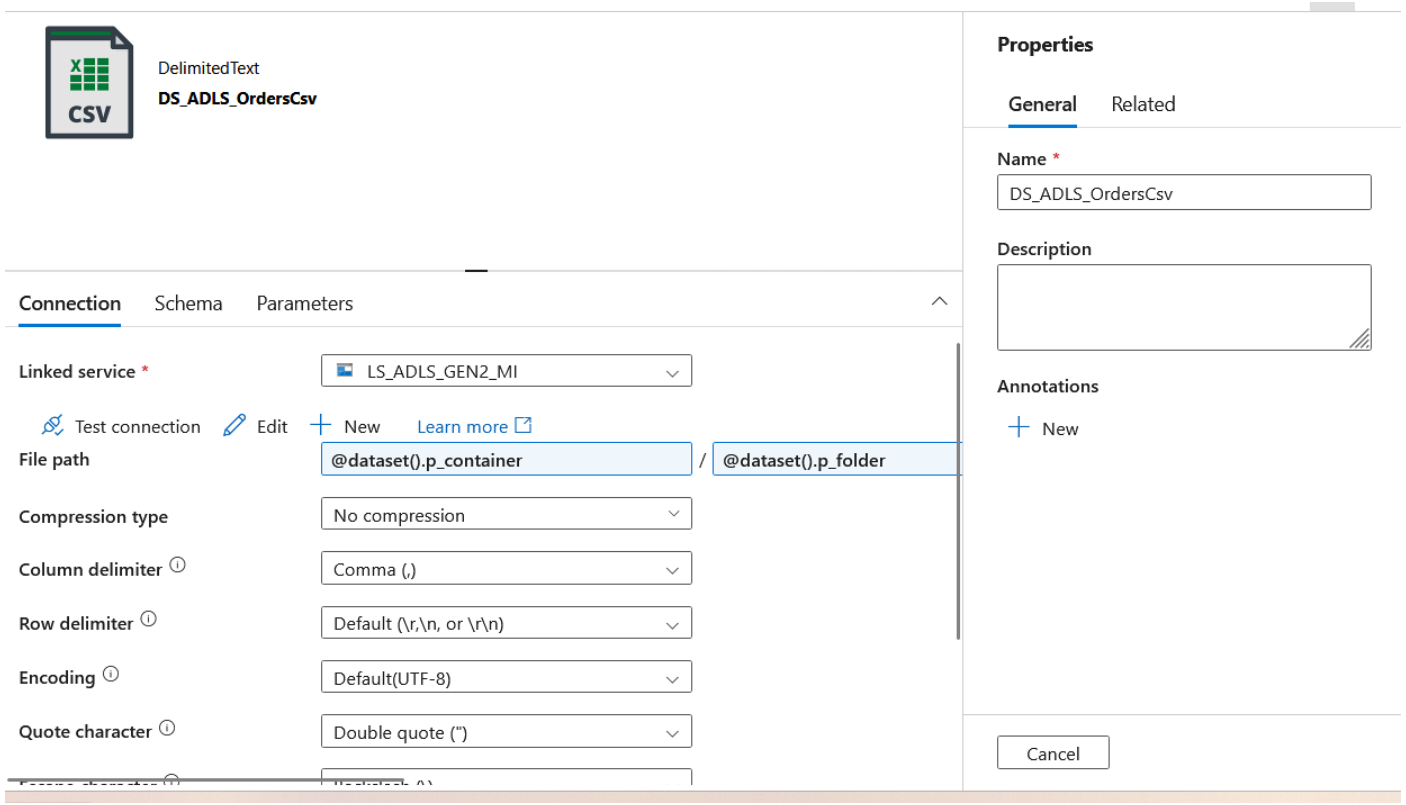
Configure DS_ADLS_OrdersCsv



Parameterized file-path properties for the ADLS CSV dataset.

What this proves: Pipeline parameters can resolve the same dataset to different folders/files.

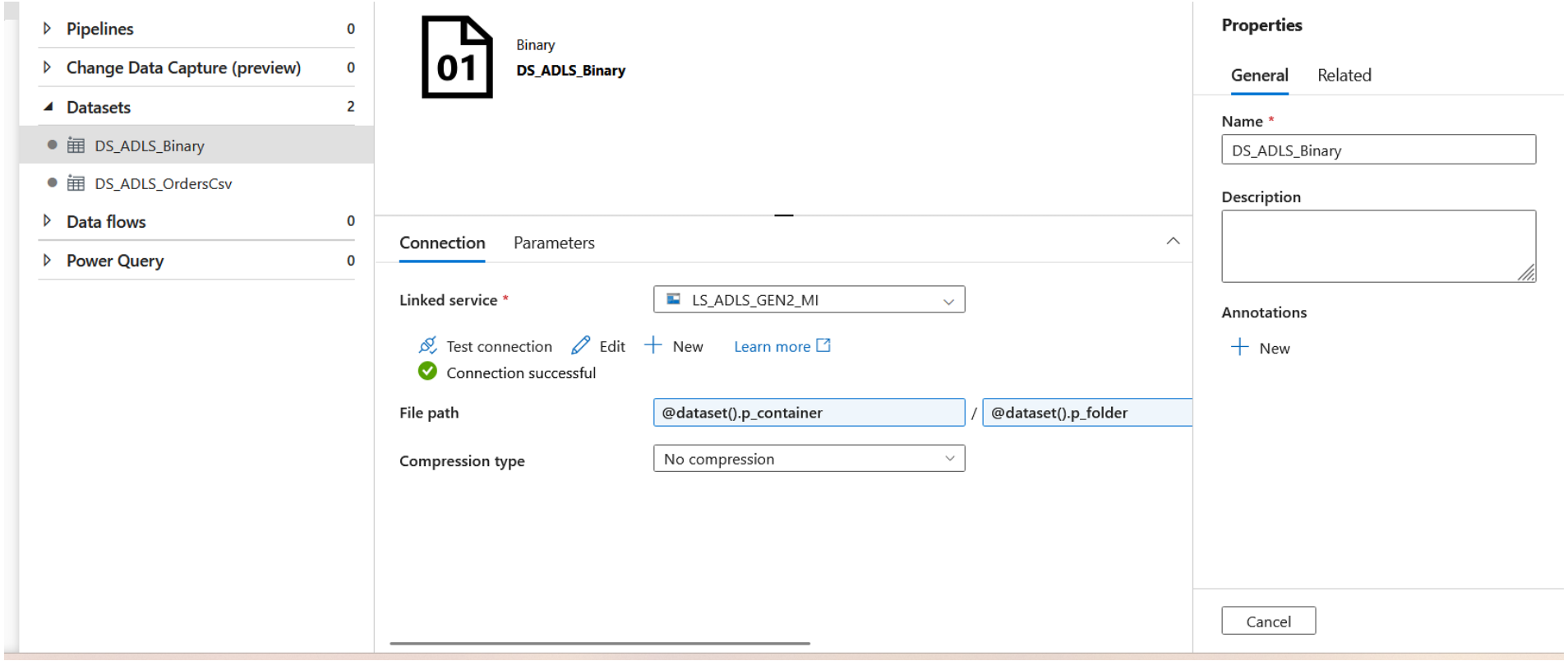
DelimitedText connection settings



DelimitedText ADLS orders dataset connection and format settings.

What this proves: First-row-as-header and comma-delimited configuration must match the input file.

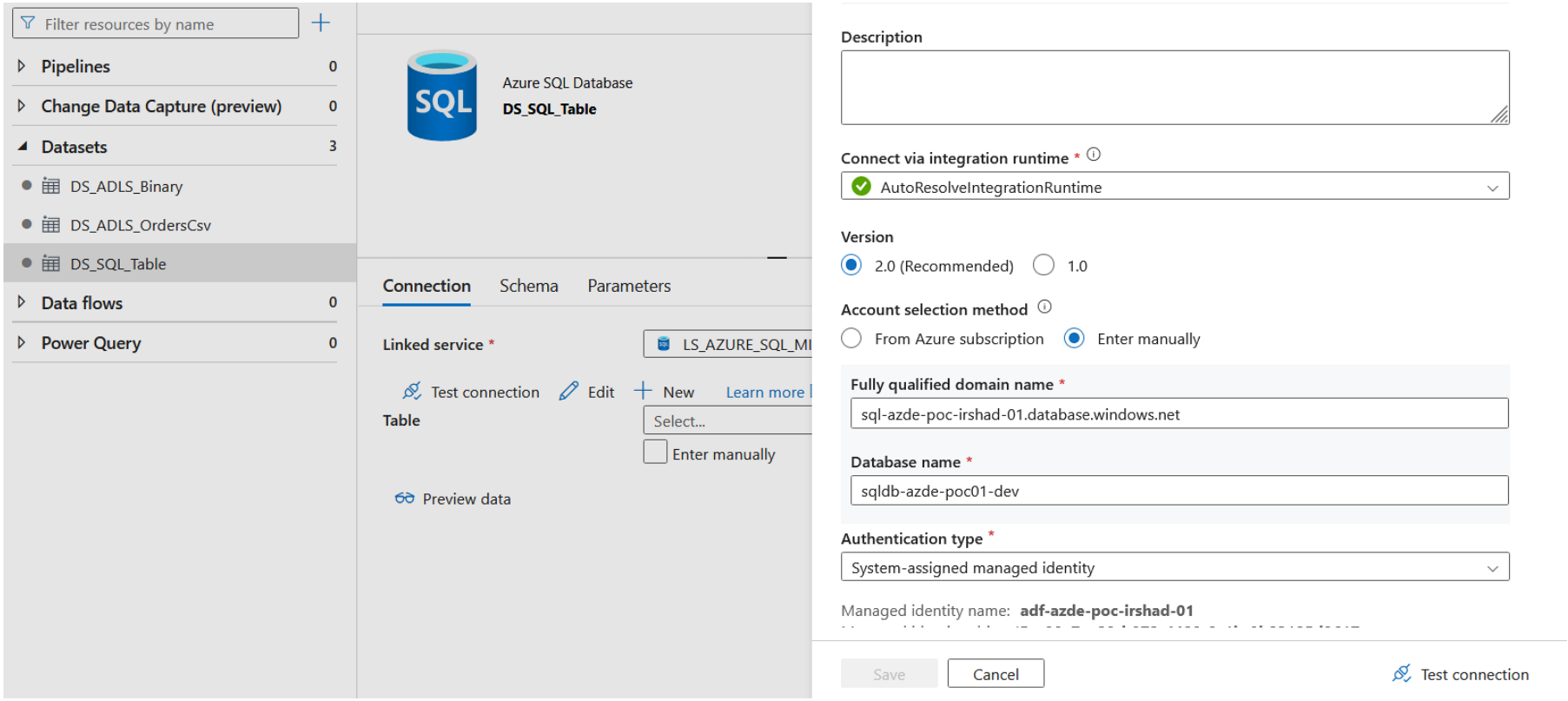
Configure the Binary ADLS dataset



Binary ADLS dataset settings for archive/delete operations.

What this proves: Binary copy preserves the raw source bytes without interpreting CSV columns.

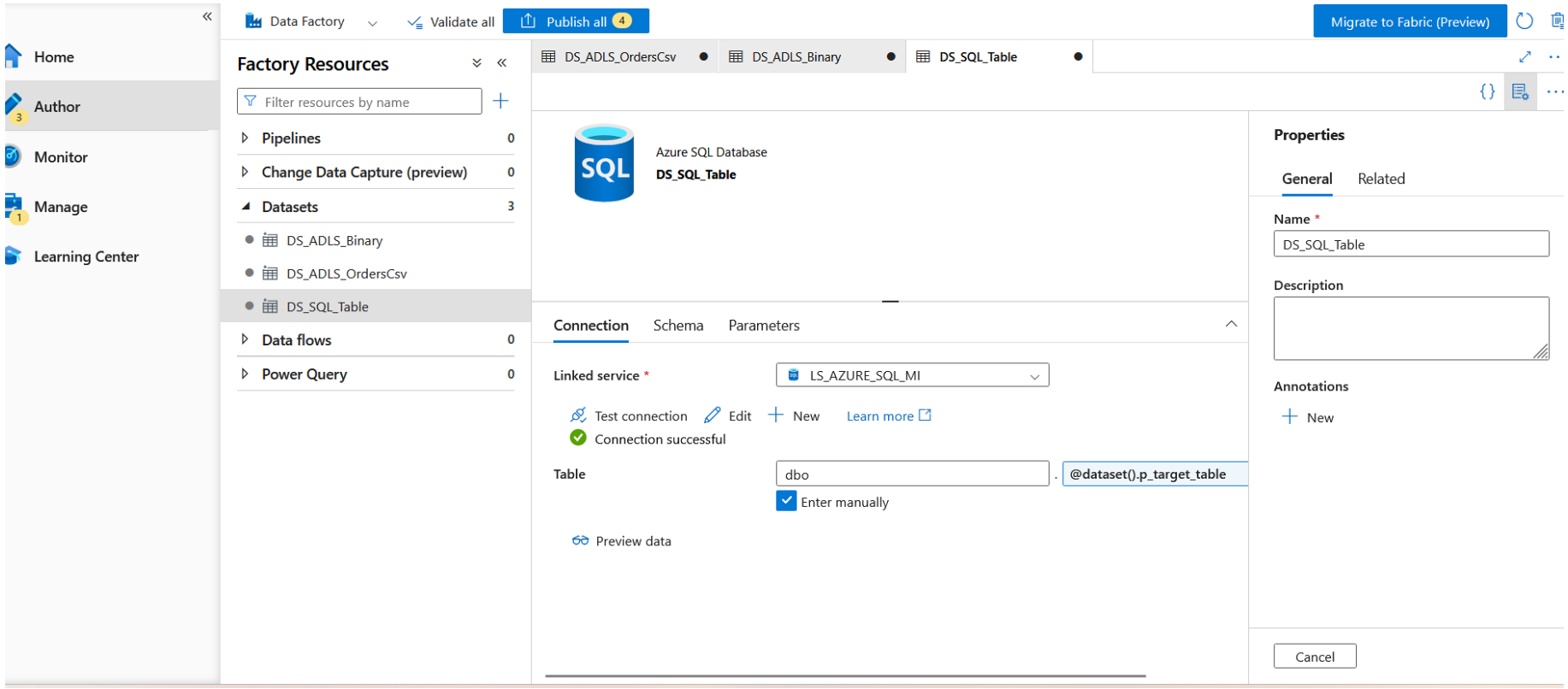
Configure Azure SQL connectivity in ADF



Azure SQL linked-service configuration using Managed Identity.

What this proves: This connection is reused by the SQL dataset and Script/Stored Procedure activities.

Configure DS_SQL_Table



Parameterized Azure SQL table dataset.

What this proves: The pipeline passes orders_stg as p_target_table.

8. Build PL_INGEST_ORDERS_BATCH

p_container	String, default landing
p_folder	String, default orders/2026/08/28
p_file	String, default orders_001.csv
p_target_table	String, default orders_stg

Activity order

- 1. Get_Metadata_File - check Exists, Size and Last modified.
- 2. If_File_Exists - True continues; False goes to Fail_File_Not_Found.
- 3. Script_Log_Start - insert IN_PROGRESS row.
- 4. Copy_Orders_To_Staging - CSV to dbo.orders_stg; truncate staging before copy.
- 5. SP_Merge_Orders - execute dbo.usp_merge_orders.
- 6. Copy_Archive_File - landing to archive using Binary dataset.
- 7. Delete_Landing_File - remove processed landing copy.
- 8. Script_Log_Success - update the ETL log to SUCCESS.

Pipeline expressions and SQL snippets

If condition, failure message and pre-copy

```
If condition
@activity('Get_Metadata_File').output.exists

Fail message
@concat('File does not exist: ', pipeline().parameters.p_container, '/',
        pipeline().parameters.p_folder, '/', pipeline().parameters.p_file)

Copy pre-copy script
TRUNCATE TABLE dbo.orders_stg;
```

Script_Log_Start

```
Script_Log_Start
INSERT INTO dbo.etl_file_log (file_name, file_size_bytes, status, started_at)
VALUES ('@{pipeline().parameters.p_file}',
        @{activity('Get_Metadata_File').output.size},
        'IN_PROGRESS', SYSUTCDATETIME());
```

Pipeline success logging

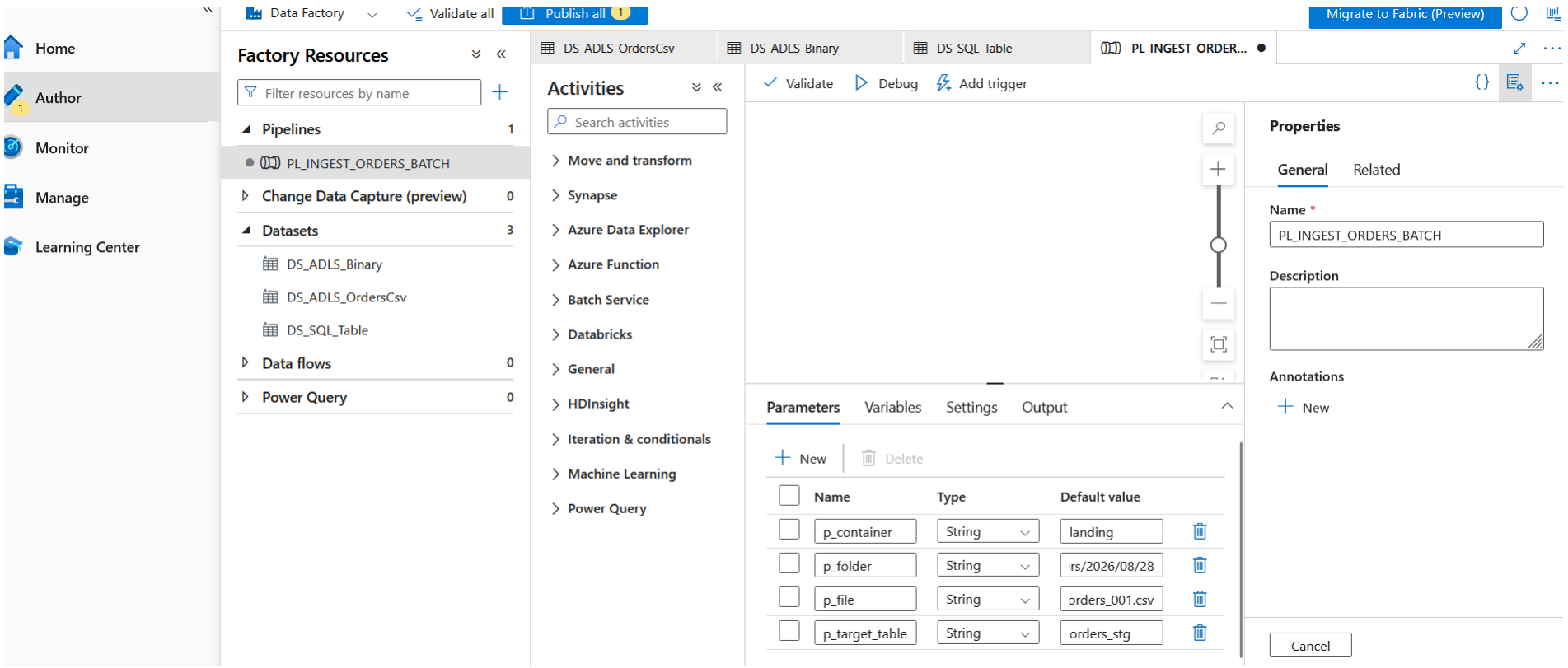
Script_Log_Success

```
UPDATE dbo.etl_file_log
SET status = 'SUCCESS',
    rows_inserted = (SELECT COUNT(1) FROM dbo.orders
                     WHERE source_file = '@{pipeline().parameters.p_file}'),
    rows_updated = 0,
    completed_at = SYSUTCDATETIME()
WHERE file_name = '@{pipeline().parameters.p_file}'
  AND status = 'IN_PROGRESS';
```

Dependency rule

Each downstream activity must depend on the successful completion of the preceding required activity. Archive only after MERGE succeeds; delete landing only after archive succeeds.

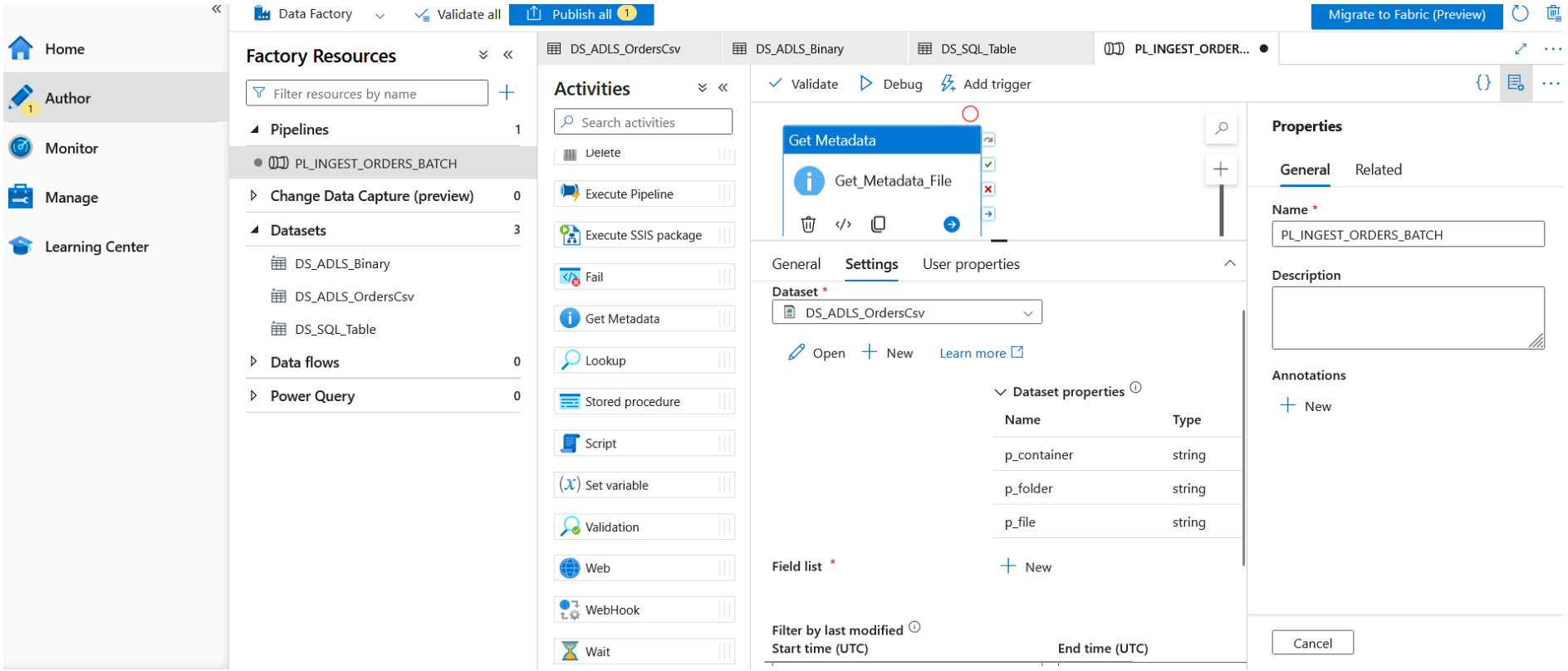
Pipeline parameters



PL_INGEST_ORDERS_BATCH pipeline parameters.

What this proves: The default values point to the manual landing file and staging table.

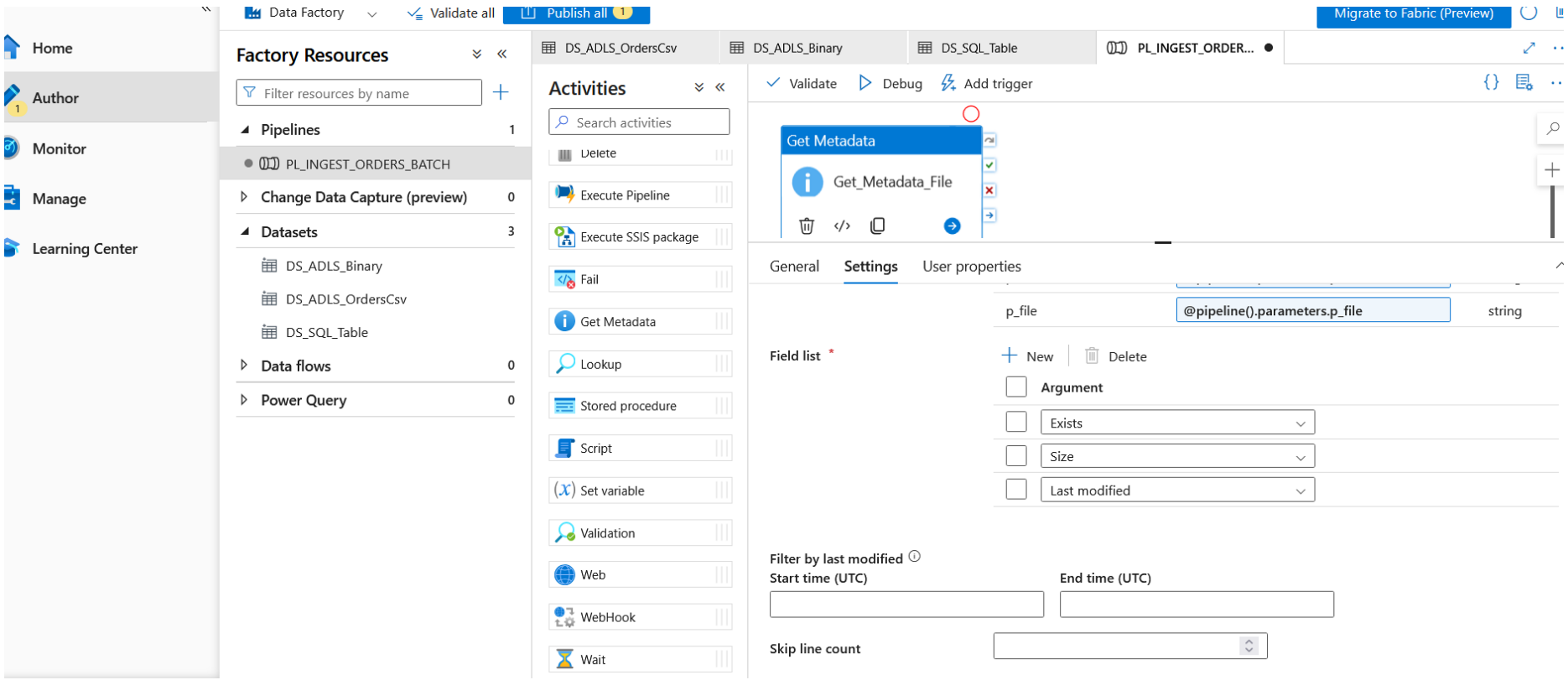
Get Metadata activity



Get_Metadata_File activity configuration.

What this proves: It reads the parameterized CSV dataset.

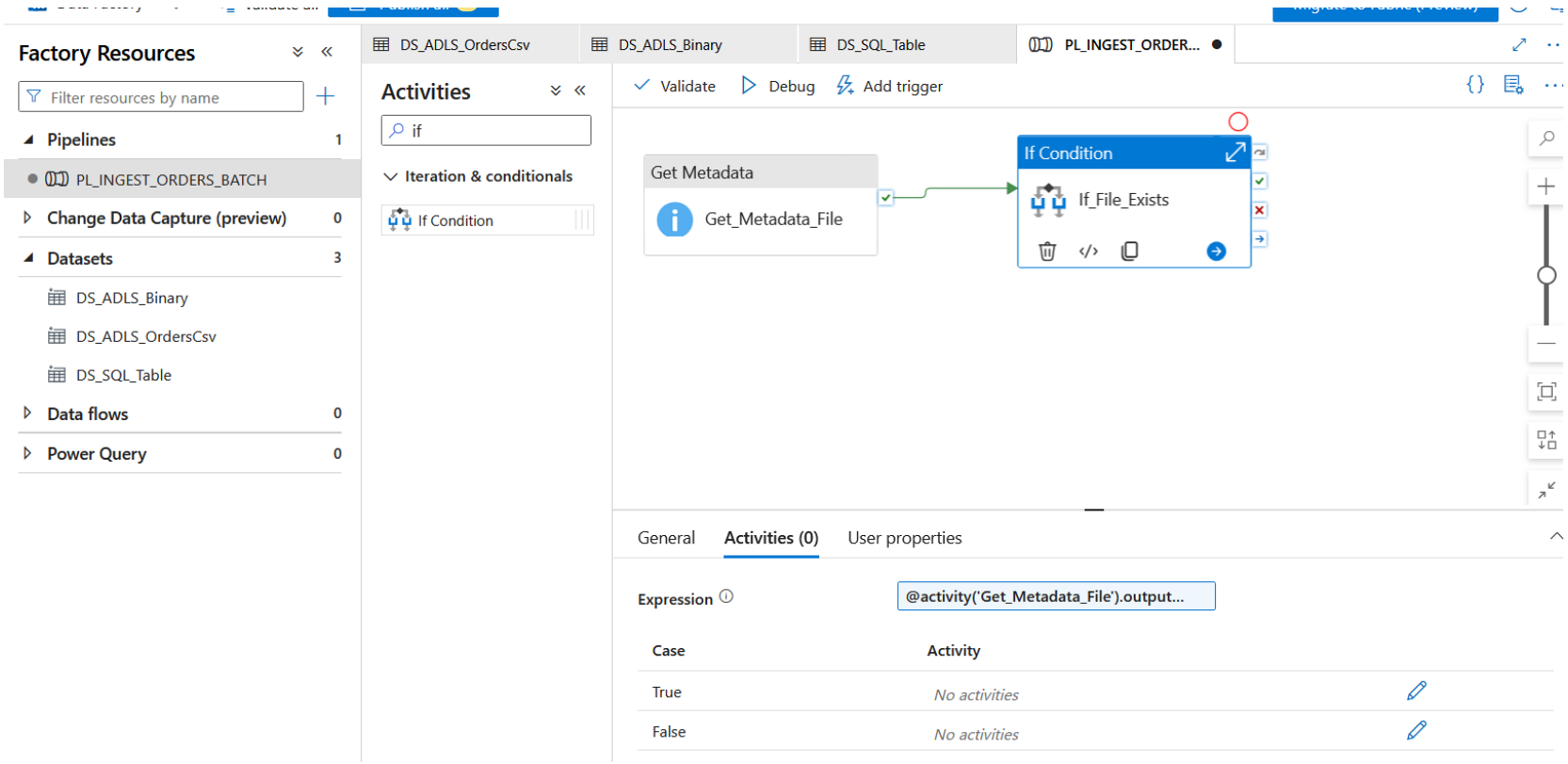
Get Metadata fields



Get Metadata field-list settings.

What this proves: Exists, Size and Last modified provide both control and audit information.

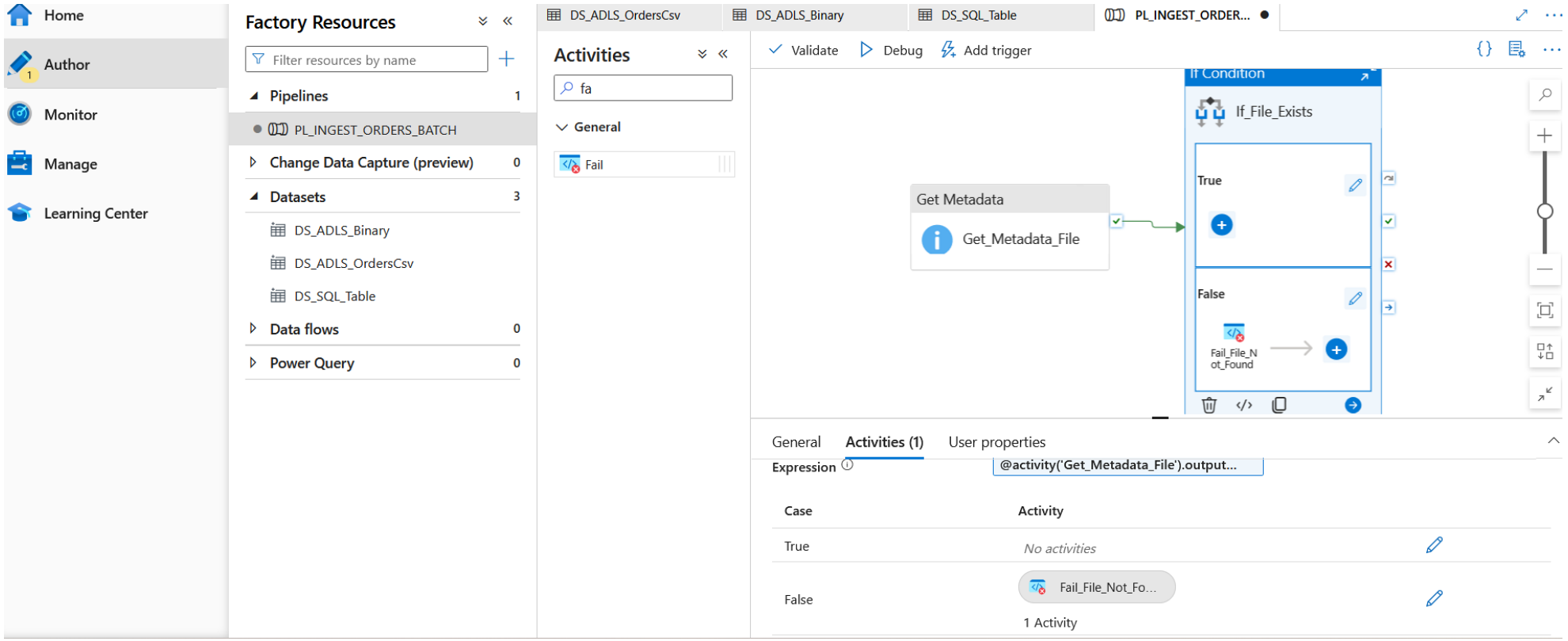
If_File_Exists activity



If condition connected after Get Metadata.

What this proves: The pipeline explicitly branches on file existence.

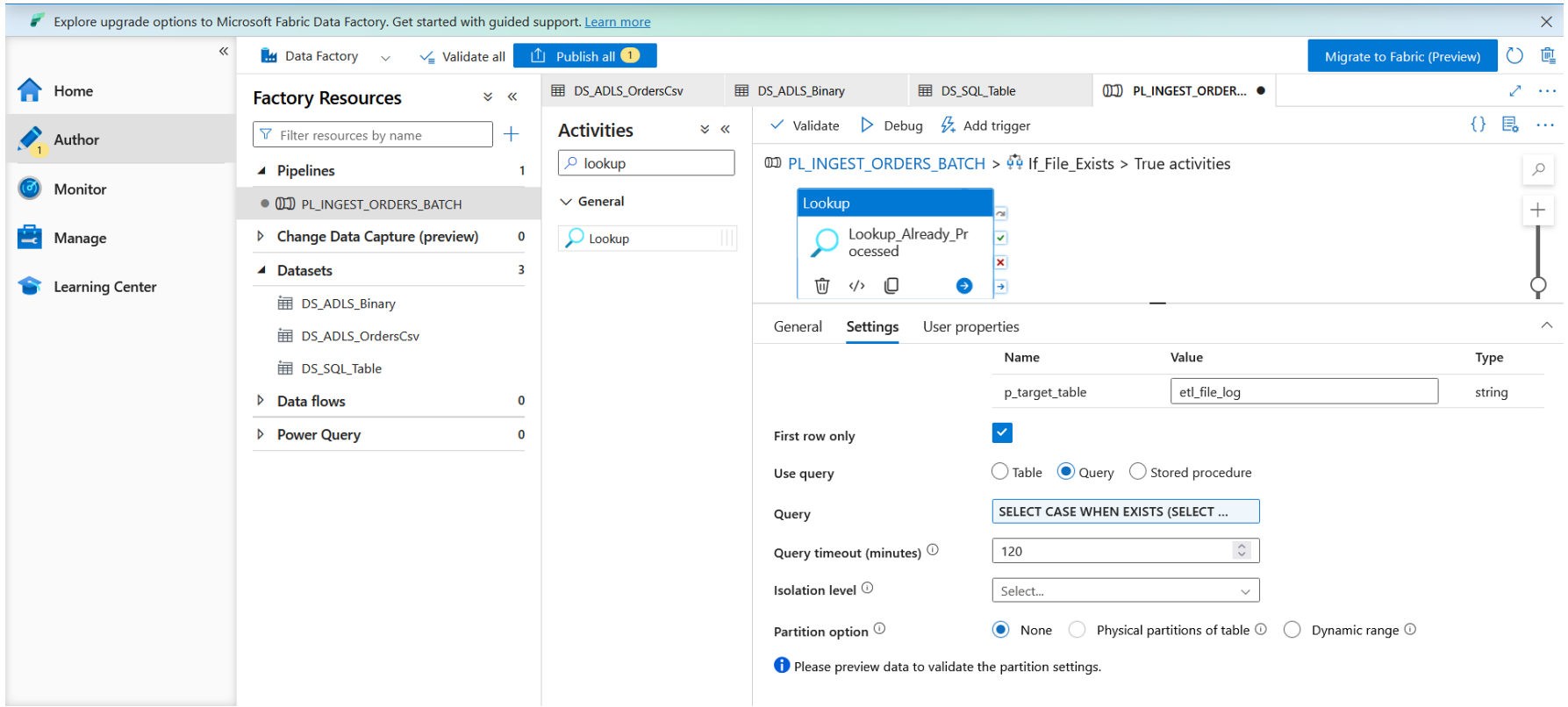
Fail_File_Not_Found branch



False branch with a Fail activity for missing input.

What this proves: A missing source is treated as a controlled failure instead of a silent success.

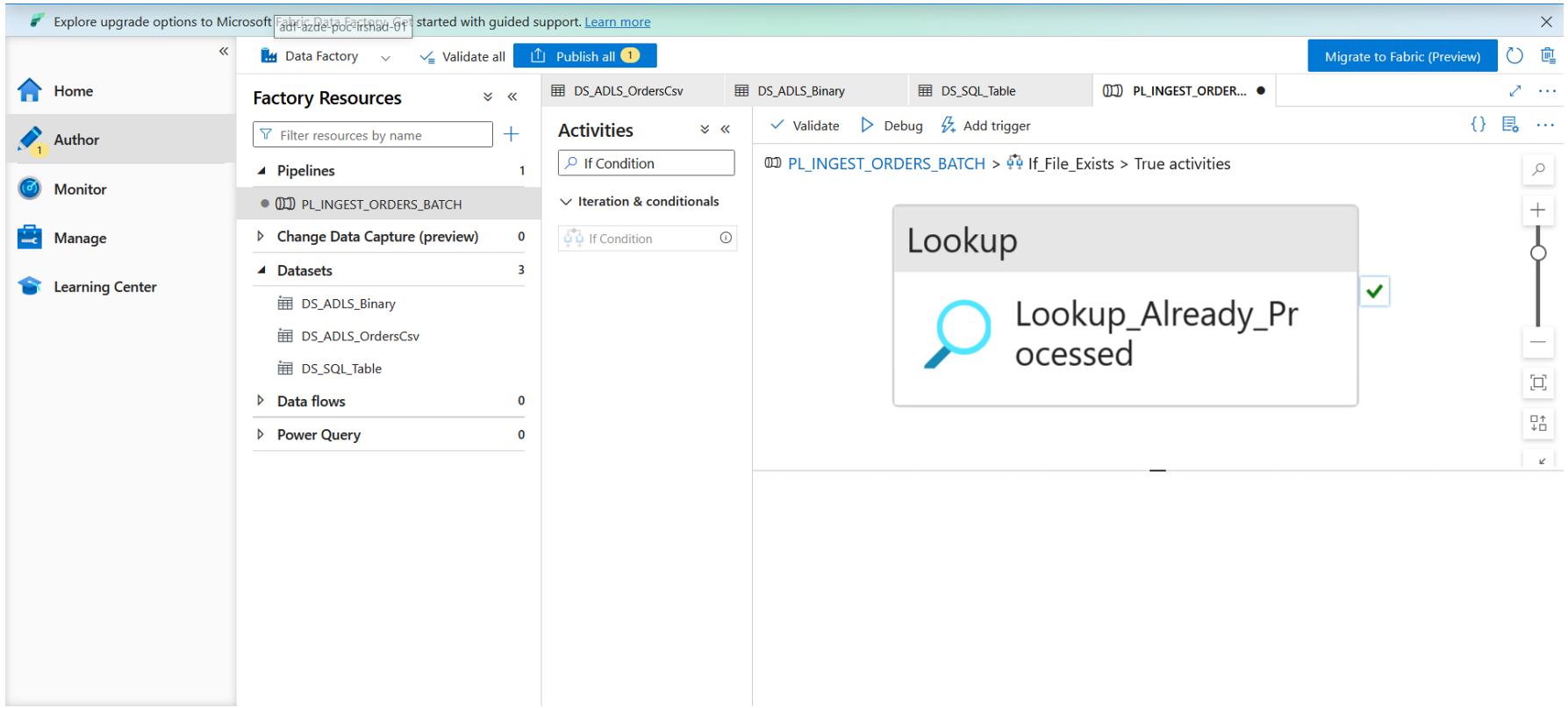
Lookup/query authoring evidence



ADF lookup/query configuration used during pipeline authoring.

What this proves: The repository advanced variant uses lookup-based processed-file control; the manual pipeline can remain simpler.

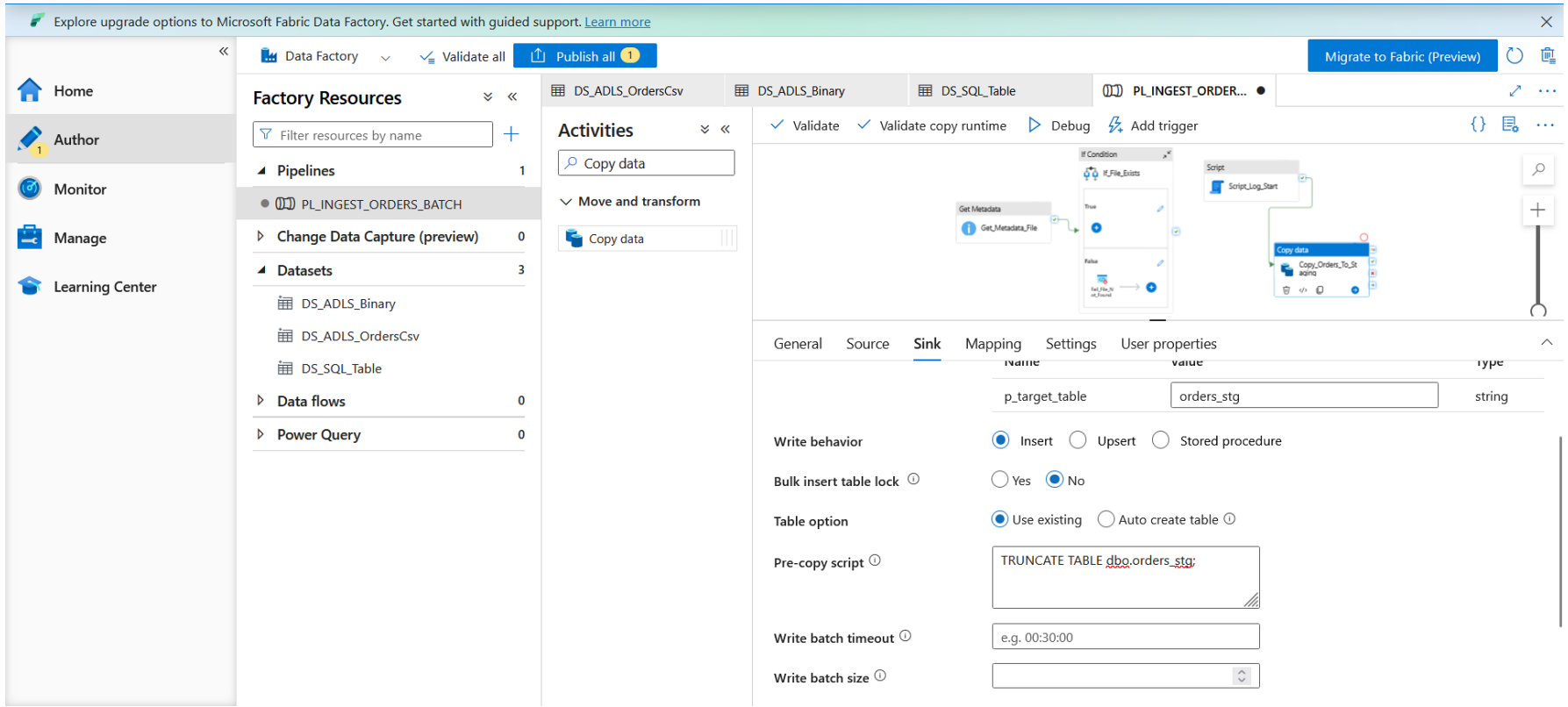
Lookup activity on pipeline canvas



Lookup activity visible on the ADF pipeline canvas.

What this proves: This evidence belongs to the enhanced file-control authoring path.

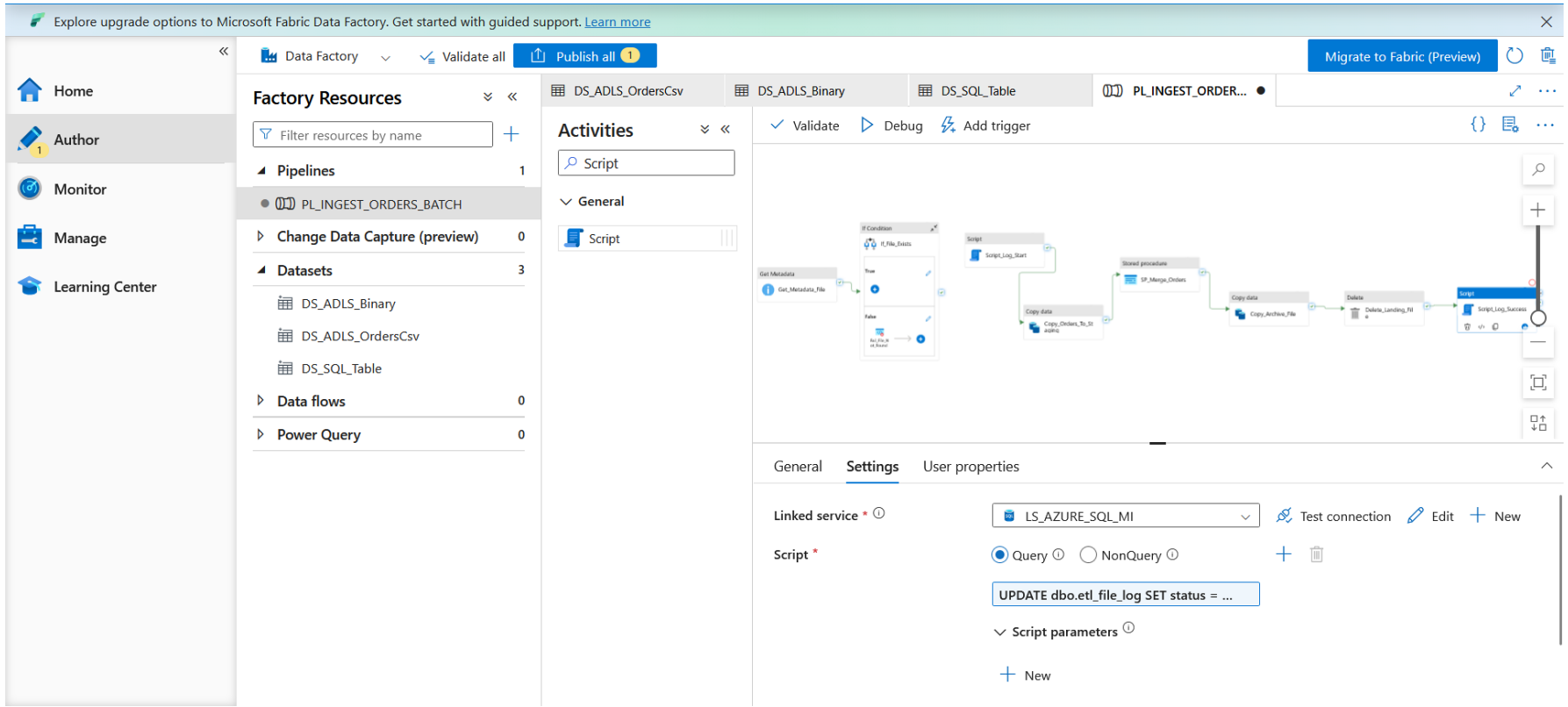
Copy CSV to SQL staging



Copy Activity configuration including staging/pre-copy behavior.

What this proves: TRUNCATE before copy makes each manual load start with a clean staging table.

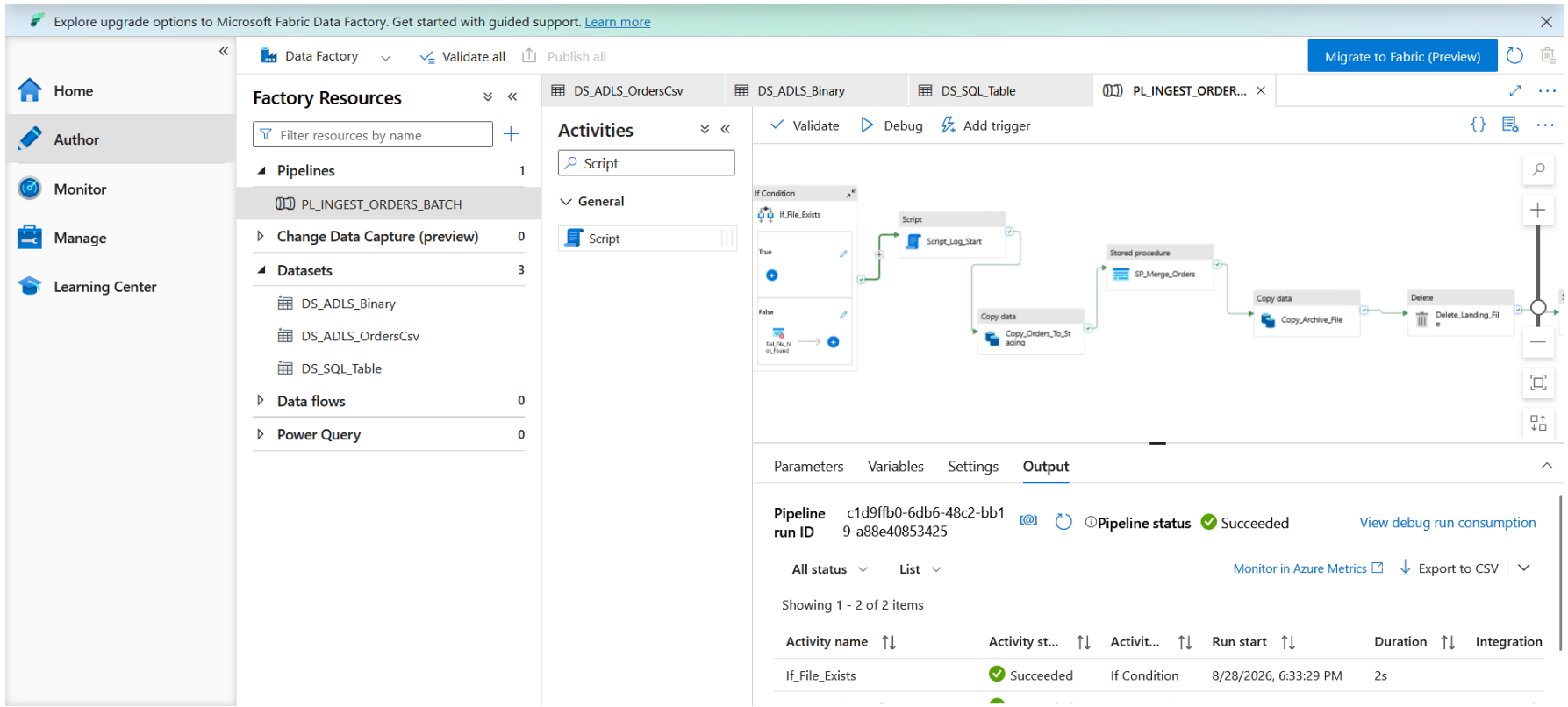
ADF Script activity



Script/SQL activity configuration in the pipeline.

What this proves: SQL logging is performed as part of orchestration.

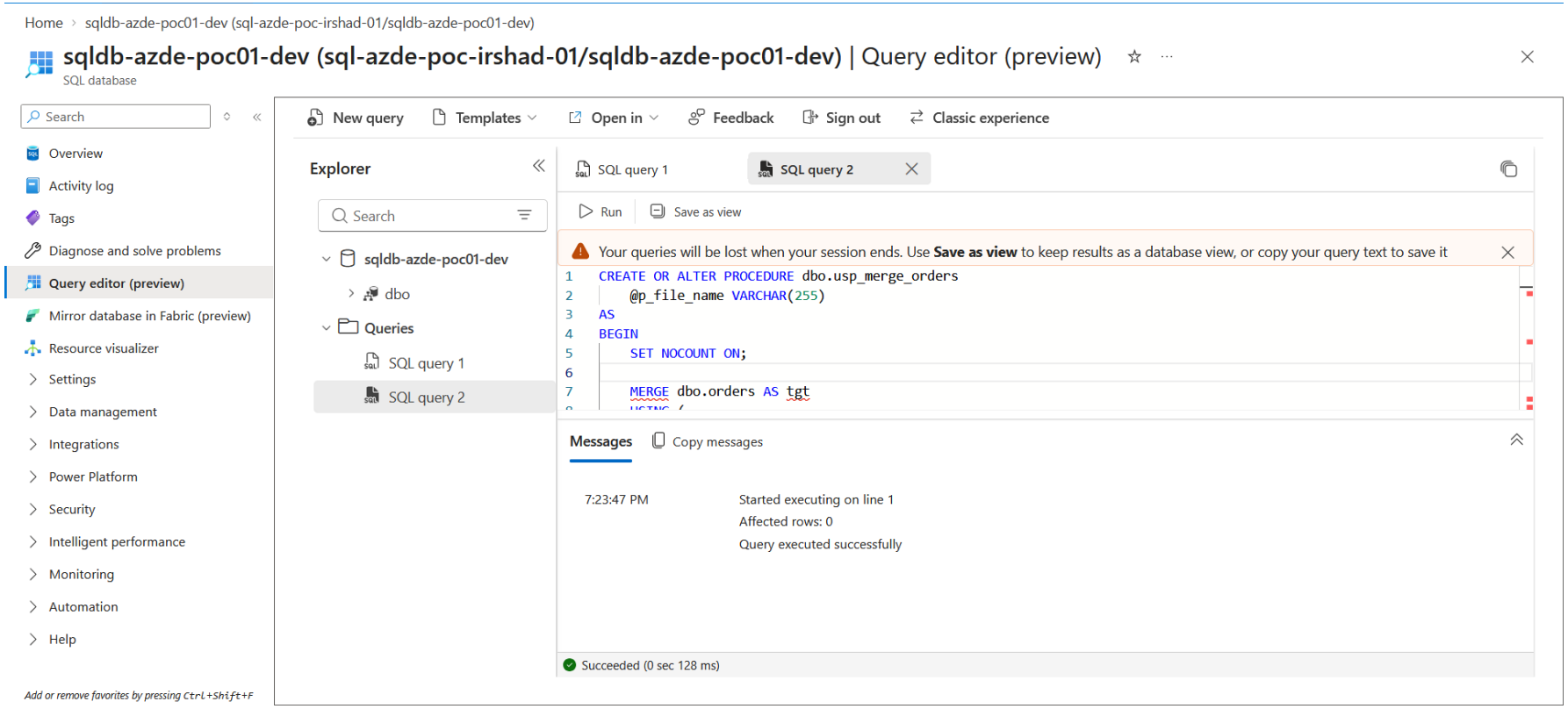
Initial pipeline debug



ADF pipeline debug/output during early execution.

What this proves: Use activity-level outputs to locate the first failing step.

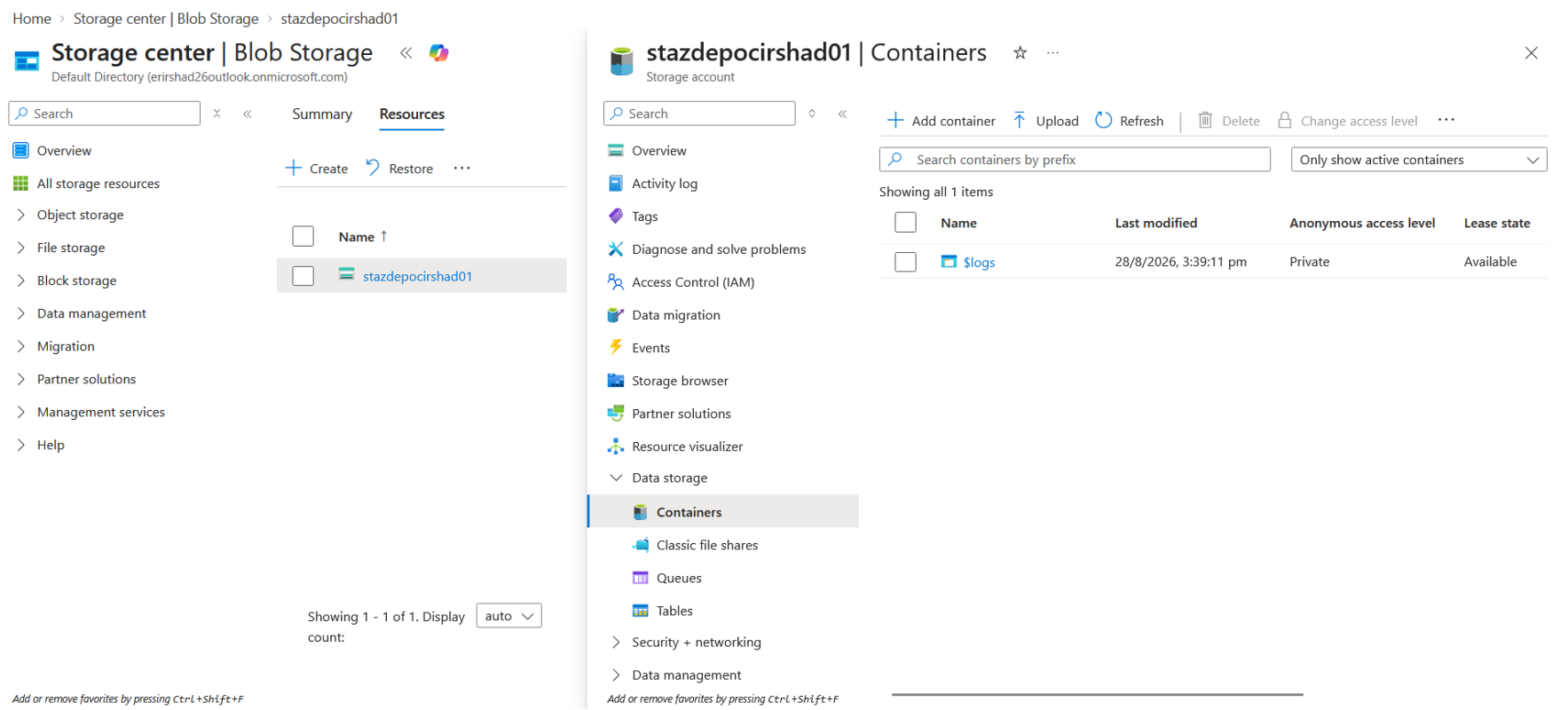
SQL Query Editor during testing



Azure SQL Query Editor used while configuring and testing database objects.

What this proves: SQL-side verification is independent evidence that the pipeline did the expected work.

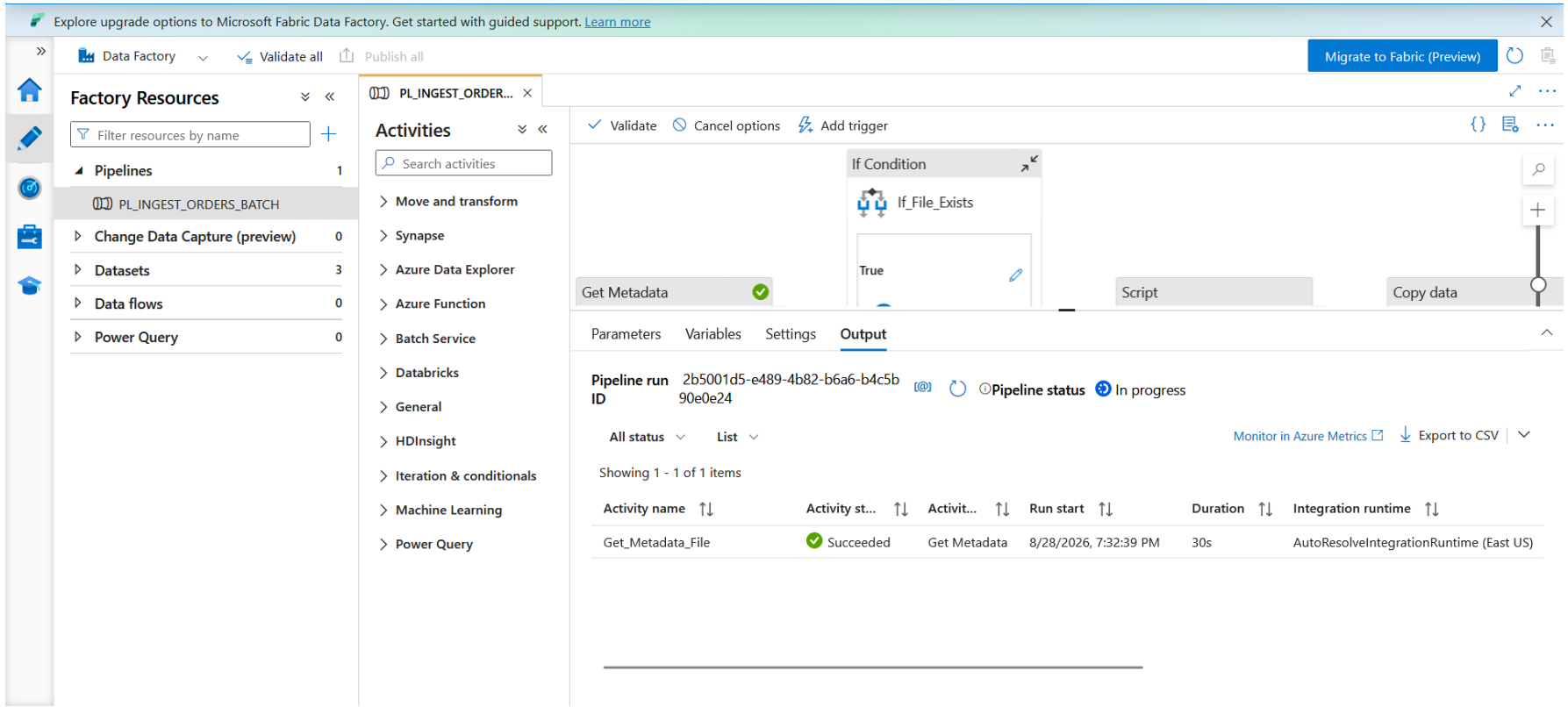
ADLS container verification



Storage/container verification during pipeline testing.

What this proves: Check landing/archive paths directly instead of relying only on pipeline status.

Get Metadata execution evidence



Get_Metadata_File succeeded during a pipeline run.

What this proves: A green activity confirms that ADF can reach the configured source path.

9. Python authentication - understand the az error

The Poetry packages azure-identity and azure-storage-file-datalake are Python SDK libraries. They do not install the az command. Azure CLI is a separate Windows application.

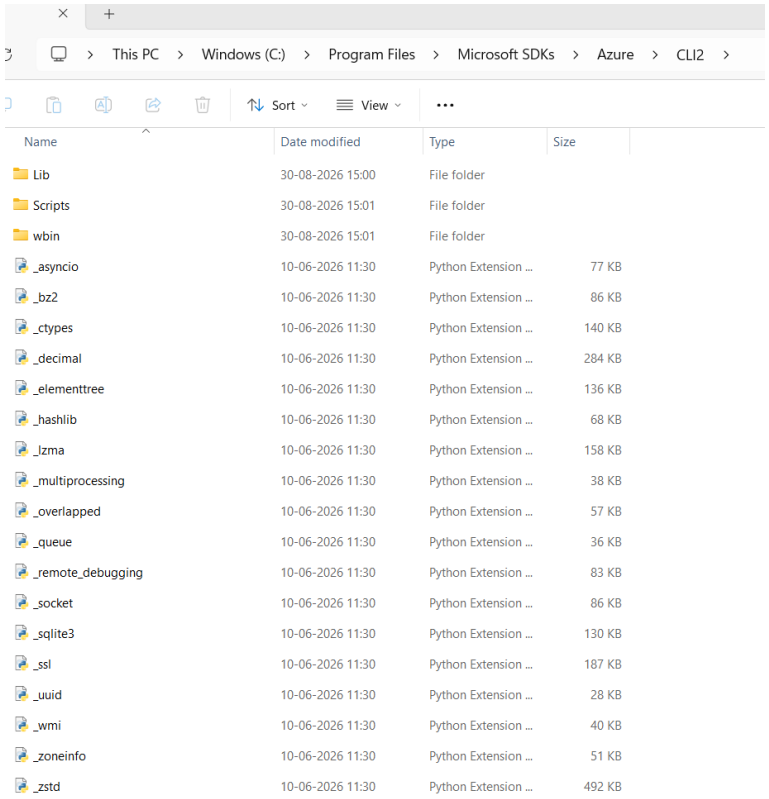
Install and verify Azure CLI

```
winget install --exact --id Microsoft.AzureCLI

# Close all CMD/PowerShell windows, then open a new terminal
az version
az login
az account show -o table
az group show --name rg-azde-poc01-dev -o table

# If PATH still looks wrong
where az
```

Observed error: 'az' is not recognized

Install Microsoft Azure CLI separately. After installation, close all terminals and reopen a new CMD/PowerShell so Windows reloads PATH. Verify with az version before attempting az login.

Azure CLI installation evidence



Windows Azure CLI installation directory after installing the CLI.

What this proves: This proves Azure CLI is a separate local installation from the Poetry Python packages.

Python environment and ADLS upload

Verify the Poetry environment

```
poetry run python --version
poetry run python -c "import azure.identity; import azure.storage.filedatalake; print('Azure Python SDK OK'
```

The completed manual pipeline expects this 5-row order_date CSV:

Manual 5-row CSV

```
order_id,customer_id,order_date,product_id,quantity,unit_price,status
1001,501,2026-08-28,10,2,25.50,Completed
1002,502,2026-08-28,12,1,100.00,Pending
1003,503,2026-08-28,15,4,15.75,Completed
1004,504,2026-08-28,10,1,25.50,Shipped
1005,505,2026-08-28,20,3,45.00,Completed
```

Upload the manual CSV to ADLS

Upload command

```
poetry run python python\upload_to_adls.py ^
  --storage-account <your-storage-account> ^
  --container landing ^
  --remote-path orders/2026/08/28/orders_001.csv ^
  --local-file data/generated/orders_001.csv ^
  --overwrite
```

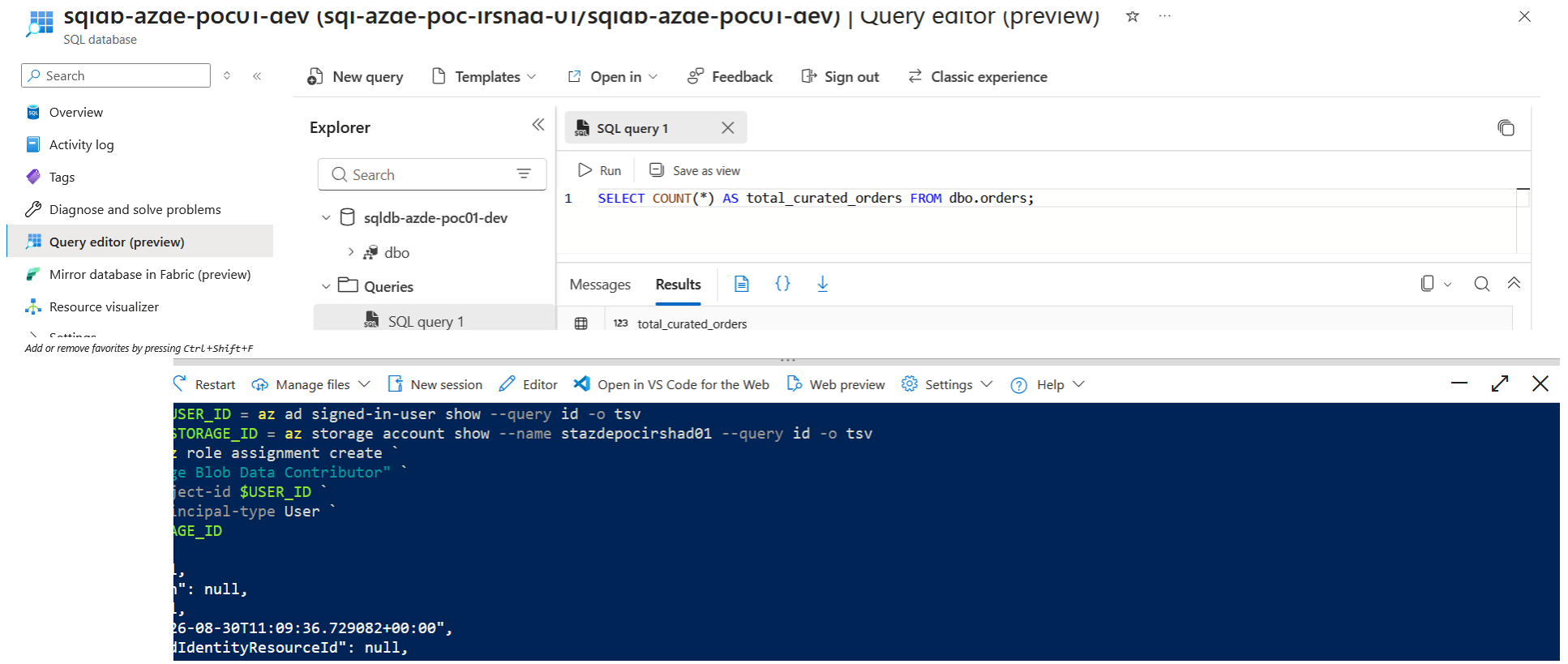
Authentication flow

az login -> Microsoft Entra token -> DefaultAzureCredential -> DataLakeServiceClient -> ADLS Gen2. No Storage key or SAS token is required in the script.

If Python returns 403

Assign Storage Blob Data Contributor to the signed-in developer user. A role assigned only to the ADF Managed Identity does not automatically authorize your local Python process.

Developer RBAC and SQL verification



Azure CLI/Cloud Shell role-assignment evidence together with SQL verification.

What this proves: The developer identity receives Storage Blob Data Contributor separately from ADF.

10. End-to-end run and verification

- Confirm orders_001.csv exists in landing/orders/2026/08/28/.
- ADF Studio -> Validate all.
- Publish all.
- Run Debug or Trigger now using the default parameters.
- Open Monitor and inspect each activity.
- Run SQL verification queries.
- Verify the raw file exists in archive and no longer exists in landing.

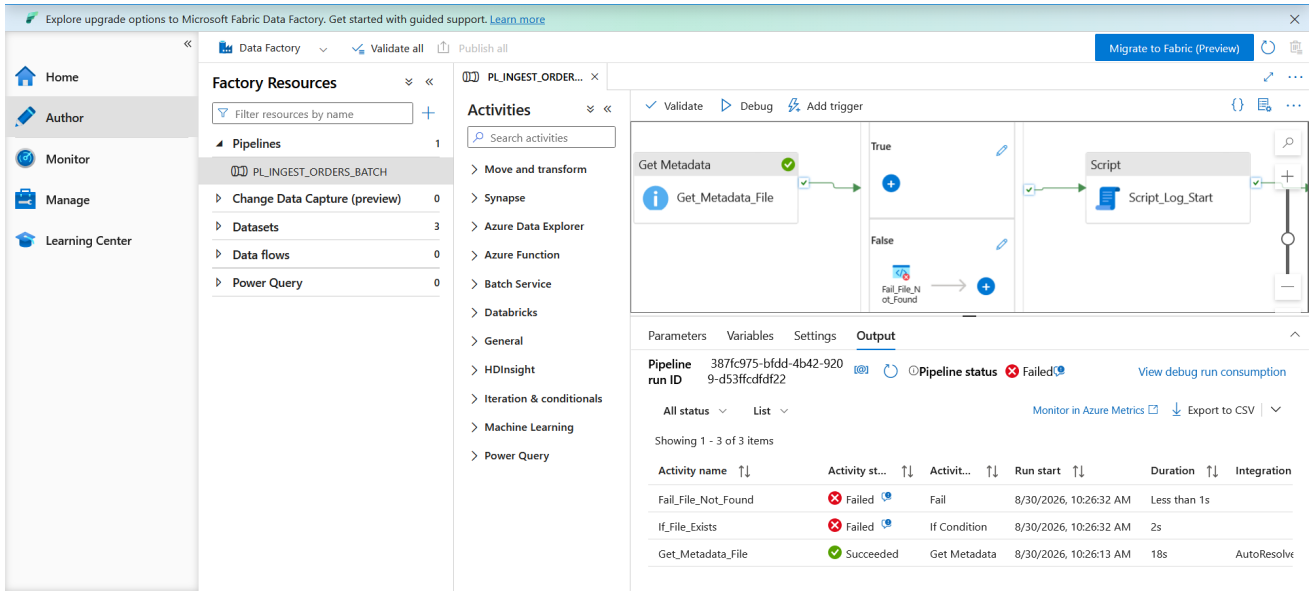
Verification queries

```
SELECT * FROM dbo.orders ORDER BY order_id;
SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;
SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

Pass condition

dbo.orders contains exactly 5 business rows and the latest ETL log row is SUCCESS.

Controlled failure - file not found



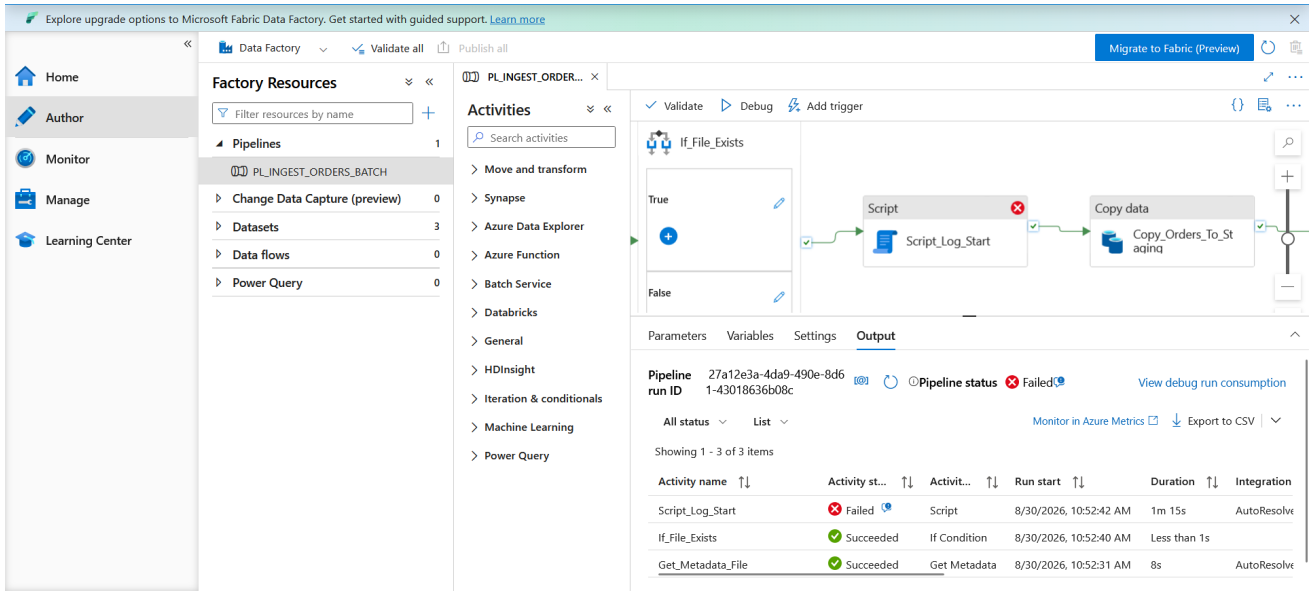
Get Metadata succeeded, but the existence condition followed the false branch and Fail_File_Not_Found failed the run.

What this proves: This is expected when the landing file is absent.

Issue and fix

Re-upload the file to the exact landing/orders/2026/08/28/orders_001.csv path, or correct p_folder/p_file. Then rerun.

Troubleshooting - Script_Log_Start failed



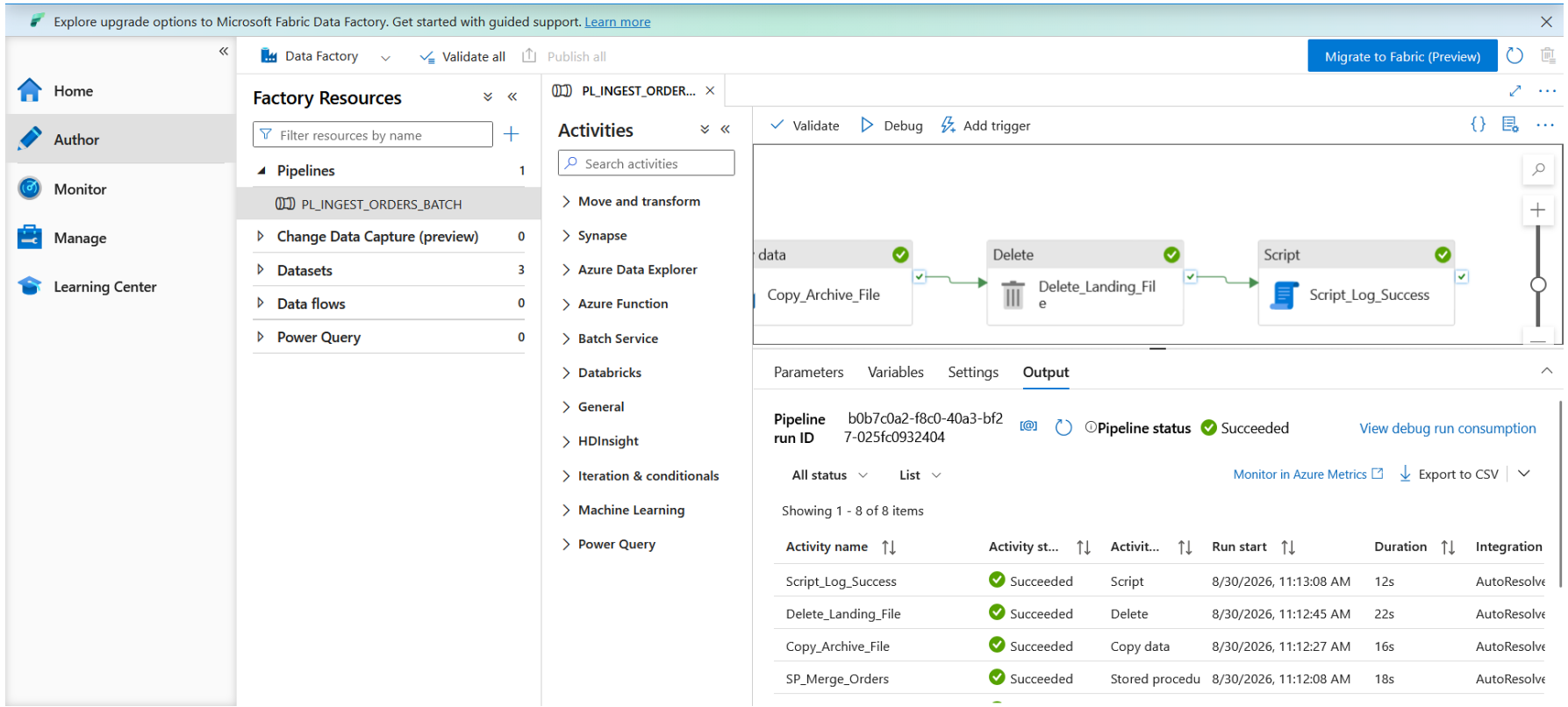
The file check passed, but Script_Log_Start failed after about 1 minute 15 seconds.

What this proves: This isolates the problem to Azure SQL connectivity/authentication rather than ADLS.

Issue and fix

Check the SQL logical-server firewall and enable the beginner-lab Azure services exception. Also verify Microsoft Entra admin and the ADF contained database user/permissions.

Successful pipeline after fixes



The pipeline completed through MERGE, archive copy, landing delete and SUCCESS logging.

What this proves: The previously failing configuration was corrected and all eight activities completed.

SQL verification evidence

SQL query 1

Run | Save as view

```
1  -- 1. Check curated/target table (should have 5 rows from orders_001.csv)
2  SELECT * FROM dbo.orders;
3
4  -- 2. Check staging table (should be truncated / 0 rows)
5  SELECT * FROM dbo.orders_stg;
6
7  -- 3. Check ETL file log table (status should be 'SUCCESS')
8  SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

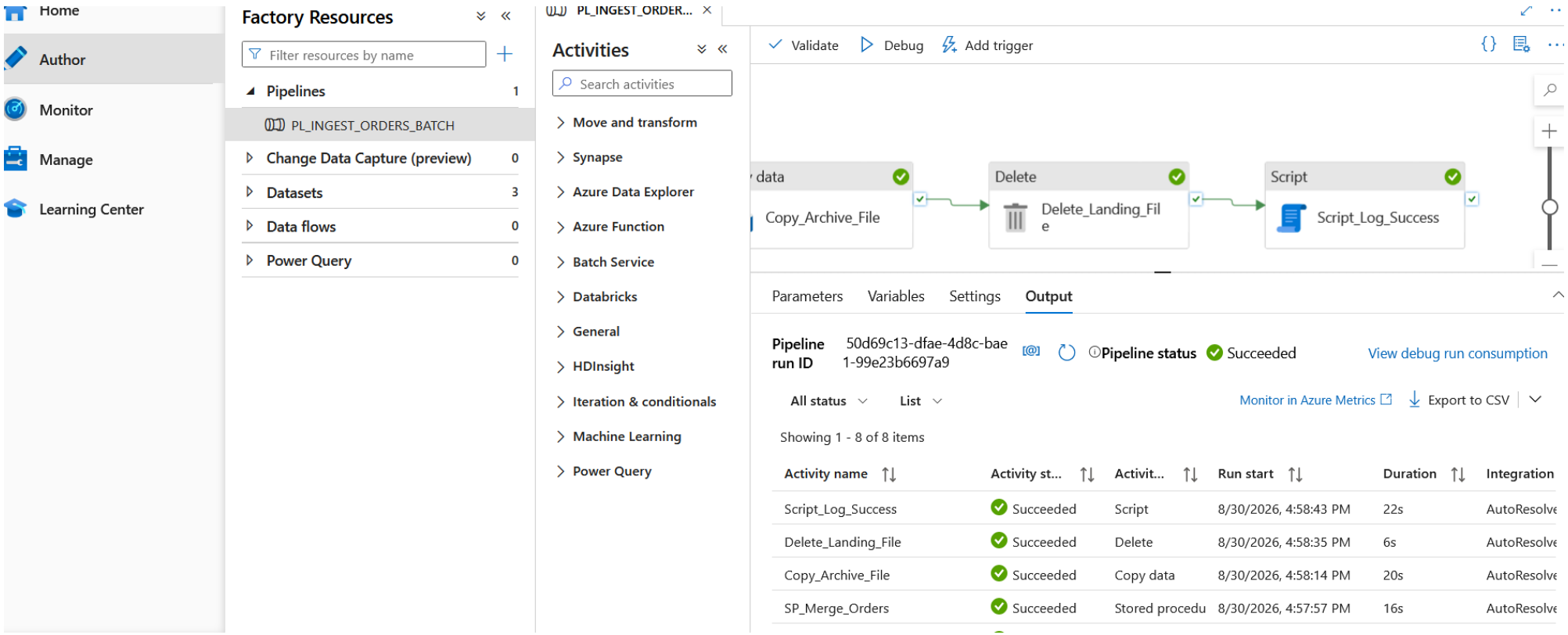
Messages | Results

	log_id	file_name	file_size_...	status	rows_ins...	rows_up...	started_at	complete...	error_me...
1	1	orders_001.csv	276	SUCCESS	5	0	2026-08-30T0...	2026-08-30T0...	*NULL*

Query Editor shows the ETL file log with SUCCESS and 5 inserted rows.

What this proves: Use SQL results as independent verification of ADF execution.

Final successful ADF run



Final pipeline evidence showing completed activities.

What this proves: This is the end-to-end success state to reproduce in a future run.

Idempotency, archive and missing-file tests

Idempotency

- Re-upload the same orders_001.csv to the same landing path.
- Run the pipeline again.
- Query SELECT COUNT(*) FROM dbo.orders.
- Pass: count remains 5, not 10, because MERGE matches on order_id.

Archive

- After success, archive/orders/2026/08/28/orders_001.csv exists.
- The landing copy is deleted only after archive succeeds.

Missing file

- Run again without re-uploading.
- The pipeline should follow Fail_File_Not_Found. This is a controlled negative test.

RBAC controlled-failure test

- Re-upload the test file.
- Temporarily remove Storage Blob Data Contributor from the ADF Managed Identity only.
- Run ADF and capture the 403/authorization failure.
- Restore the role assignment.
- Retry the same pipeline and verify success.

What this test proves

The pipeline is genuinely authorized through Managed Identity + RBAC. Removing authorization produces an observable failure; restoring the correct permission allows safe retry.

11. Errors encountered and fixes - Part 1

'az' is not recognized	azure-identity is a Python SDK, not Azure CLI. Install Microsoft.AzureCLI separately, reopen the terminal, and verify az version.
Azure SQL region/capacity restriction	Choose a region/configuration available to the subscription. Retry SQL server/database creation there.
Script_Log_Start timeout / failure	Check SQL server firewall and the beginner-lab Allow Azure services and resources option. Then verify Microsoft Entra SQL configuration.
SQL Managed Identity login fails	Set a Microsoft Entra admin, connect to the target database, CREATE USER for the exact ADF resource name, then grant required database permissions.

Errors encountered and fixes - Part 2

Get Metadata says file absent	Verify p_container, p_folder and p_file exactly match the ADLS path. For this manual run: landing/orders/2026/08/28/orders_001.csv.
Python upload returns 403	Assign Storage Blob Data Contributor to the signed-in developer identity, not only to ADF.
ADF storage access returns 403	Assign or restore Storage Blob Data Contributor to the ADF system-assigned Managed Identity.
Duplicate rerun concern	The curated table primary/business key is order_id and MERGE updates/matches, so the manual 5-row sample remains 5 rows.
Archive copy succeeds but landing remains	Verify Delete activity dependency, original landing dataset parameters and delete permission.

Two subtle lessons that prevent false debugging

1. Manual staging table may still contain 5 rows

The manual stored procedure shown in the setup does not delete staging rows after MERGE. The Copy Activity truncates staging before the next load. Therefore dbo.orders_stg can legitimately contain 5 rows immediately after success. Do not treat that alone as a pipeline failure.

2. order_date and order_ts are different variants

The manual pipeline uses order_date. The repository generate_orders.py uses the advanced order_ts schema. Do not upload the advanced generator output into the manual mapping unless you intentionally update SQL tables, datasets and mappings.

12. Repository files - what each part is for

python/upload_to_adls.py	Use with the completed manual POC. Uploads a local CSV using DefaultAzureCredential.
python/generate_orders.py	Advanced 30-row generator with deliberate bad rows and order_ts schema.
python/inspect_orders.py	Validates the advanced generated file locally.
sql/	Advanced tables, rejects, file log, watermark and transaction-safe MERGE.
adf/	Sanitized advanced ADF reference JSON and dataset/linked-service templates.
infra/terraform/	Infrastructure-as-code recreation of the Azure foundation.
docs/	Security, monitoring, troubleshooting, validation, interview and cleanup reference.

Manual vs advanced repository variant

Completed manual variant	5-row CSV; order_date; simple ETL log; pre-copy TRUNCATE; MERGE on order_id; archive/delete; SUCCESS logging.
Advanced repository variant	30-row CSV; order_ts; one type-invalid row; one business-invalid row; orders_rejects; processed-file control; watermark; per-file/run staging cleanup; duplicate branch.

Recommended next upgrade

After you can reproduce the manual POC without help, migrate intentionally to the advanced schema and verify the 30 -> 28 curated-row expectation. Treat this as a second lab, not an invisible change to the completed manual flow.

13. Terraform recreation - what it creates

azurerm_resource_group	POC Resource Group
azurerm_storage_account	StorageV2 with hierarchical namespace and TLS 1.2
azurerm_storage_container	landing, archive and quarantine
azurerm_data_factory	ADF with system-assigned identity
azurerm_role_assignment	ADF -> Storage Blob Data Contributor

Terraform recreation - what it creates - Continued

azurerm_key_vault	RBAC-enabled Key Vault
azurerm_mssql_server / database	Azure SQL logical server and small lab database
azurerm_mssql_firewall_rule	Allow Azure services plus optional developer IP
azurerm_log_analytics_workspace	Optional diagnostics workspace

Do not collide with the manual environment

Use a second Resource Group and unique globally scoped names, delete the manual lab first, or deliberately import existing resources. A normal terraform apply cannot simply take ownership of already-existing matching resources.

Terraform - prerequisite and variable workflow

Terraform prerequisites

```
terraform -version
az login
az account show -o table

cd infra/terraform
copy terraform.tfvars.example terraform.tfvars
```

- Use unique Storage Account, Data Factory, SQL Server and Key Vault names.
- Keep terraform.tfvars local; do not commit real SQL admin credentials.
- Set client_ip only when you need direct SQL administration from the laptop.
- Set uploader_object_id only if Terraform should grant your developer identity the ADLS data role.
- For a clean practice environment, use a new Resource Group such as rg-azde-poc01-tf-dev.

Terraform - init, validate, plan and apply

Terraform commands

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
```

- Read the plan before applying.
- Confirm the intended subscription and Resource Group.
- Type the Terraform confirmation only after reviewing the resource list.
- After apply, inspect outputs and verify the actual resources in Azure Portal.

Terraform - required post-deployment manual steps

- Set a Microsoft Entra administrator on the Azure SQL logical server.
- Connect to the database as that administrator.
- Run the SQL schema and merge scripts that match the ADF mapping you intend to deploy.
- Create the ADF contained database user.
- Author/import ADF linked services, datasets and pipeline using the same schema variant.
- Test connections and run the same end-to-end validation suite.

Networking note

The Terraform lab uses the Azure SQL AllowAzureServices firewall rule for the beginner public-endpoint design. The hardened extension is ADF Managed VNet + Private Endpoints/Private Link.

Terraform - verification and destroy

Verify and destroy

```
terraform output
az resource list --resource-group <terraform-resource-group> -o table

# Before cleanup
terraform plan -destroy
terraform destroy
```

Cleanup rule

If Terraform created the environment and state is intact, use terraform destroy. For an isolated manual lab, deleting the entire Resource Group is the simplest lifecycle cleanup.

Terraform source - versions.tf

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

Terraform source - variables.tf

```
variable "subscription_id" {
  description = "Azure subscription ID. Prefer ARM_SUBSCRIPTION_ID environment variable when possible."
  type        = string
  default     = null
}

variable "location" {
  description = "Azure region for POC resources."
  type        = string
  default     = "centralindia"
}

variable "resource_group_name" {
  type    = string
  default = "rg-azde-poc01-dev"
}

variable "storage_account_name" {
  description = "Globally unique, lowercase letters/numbers only."
  type        = string
}
```

Terraform source - variables.tf - Continued

```
variable "data_factory_name" {
  description = "Globally unique Data Factory name."
  type        = string
}

variable "sql_server_name" {
  description = "Globally unique Azure SQL logical server name (without .database.windows.net)."
  type        = string
}

variable "sql_database_name" {
  type    = string
  default = "sqldb-azde-poc01-dev"
}

variable "sql_admin_login" {
  description = "Bootstrap SQL admin login. Do not commit the real value."
  type        = string
  sensitive   = true
}

variable "sql_admin_password" {
```

Terraform source - variables.tf - Continued

```
  description = "Bootstrap SQL admin password. Do not commit the real value."
  type        = string
  sensitive   = true
}

variable "key_vault_name" {
  description = "Globally unique Key Vault name."
  type        = string
}

variable "client_ip" {
  description = "Optional public IPv4 address of your laptop for SQL admin bootstrap access."
  type        = string
  default     = null
}

variable "uploader_object_id" {
  description = "Optional Microsoft Entra object ID of your developer user. If set, grants Storage Blob Dat
  type        = string
  default     = null
}
```

Terraform source - variables.tf - Continued

```
variable "create_log_analytics" {
  description = "Create a small Log Analytics workspace for optional diagnostics practice."
  type        = bool
  default     = false
}

variable "tags" {
  type = map(string)
  default = {
    project     = "azure-poc"
    environment = "dev"
    owner       = "personal"
    autoDelete  = "true"
  }
}
```

Terraform source - main.tf

```
data "azurerm_client_config" "current" {}

resource "azurerm_resource_group" "this" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

resource "azurerm_storage_account" "this" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.this.name
  location                 = azurerm_resource_group.this.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  is_hns_enabled           = true
  min_tls_version          = "TLS1_2"

  allow_nested_items_to_be_public = false
  public_network_access_enabled   = true

  tags = var.tags
```

Terraform source - main.tf - Continued

```
}

resource "azurerm_storage_container" "landing" {
  name                  = "landing"
  storage_account_id    = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_storage_container" "archive" {
  name                  = "archive"
  storage_account_id    = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_storage_container" "quarantine" {
  name                  = "quarantine"
  storage_account_id    = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_data_factory" "this" {
  name                = var.data_factory_name
```

Terraform source - main.tf - Continued

```
  location            = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name

  identity {
    type = "SystemAssigned"
  }

  tags = var.tags
}

resource "azurerm_role_assignment" "adf_storage" {
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}

resource "azurerm_role_assignment" "developer_storage" {
  count                = var.uploader_object_id == null ? 0 : 1
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = var.uploader_object_id
}
```

Terraform source - main.tf - Continued

```
resource "azurerm_key_vault" "this" {
  name                        = var.key_vault_name
  location                    = azurerm_resource_group.this.location
  resource_group_name         = azurerm_resource_group.this.name
  tenant_id                   = data.azurerm_client_config.current.tenant_id
  sku_name                    = "standard"
  enable_rbac_authorization   = true
  soft_delete_retention_days = 7

  public_network_access_enabled = true

  tags = var.tags
}

# The main POC data path does not need a Key Vault secret because Managed Identity is preferred.
# This role lets the ADF identity read secrets later if you intentionally add a connector that requires one
resource "azurerm_role_assignment" "adf_key_vault" {
  scope                = azurerm_key_vault.this.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}
```

Terraform source - main.tf - Continued

```
resource "azurerm_mssql_server" "this" {
  name                         = var.sql_server_name
  resource_group_name          = azurerm_resource_group.this.name
  location                     = azurerm_resource_group.this.location
  version                      = "12.0"
  administrator_login          = var.sql_admin_login
  administrator_login_password = var.sql_admin_password
  minimum_tls_version          = "1.2"
  public_network_access_enabled = true

  tags = var.tags
}

resource "azurerm_mssql_database" "this" {
  name      = var.sql_database_name
  server_id = azurerm_mssql_server.this.id
  sku_name  = "Basic"
  max_size_gb = 2

  tags = var.tags
}
```

Terraform source - main.tf - Continued

```
# Beginner-lab networking compromise: lets Azure services such as ADF Azure IR reach Azure SQL.
# Replace this with managed private endpoints / Private Link in the hardened version.
resource "azurerm_mssql_firewall_rule" "allow_azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_mssql_server.this.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

resource "azurerm_mssql_firewall_rule" "developer_ip" {
  count            = var.client_ip == null ? 0 : 1
  name             = "DeveloperLaptop"
  server_id        = azurerm_mssql_server.this.id
  start_ip_address = var.client_ip
  end_ip_address   = var.client_ip
}

resource "azurerm_log_analytics_workspace" "this" {
  count               = var.create_log_analytics ? 1 : 0
  name                = "law-azde-poc01-dev"
  location            = azurerm_resource_group.this.location
```

Terraform source - main.tf - Continued

```
  resource_group_name = azurerm_resource_group.this.name
  sku                 = "PerGB2018"
  retention_in_days   = 30

  tags = var.tags
}
```

Terraform source - outputs.tf

```
output "resource_group_name" {
  value = azurerm_resource_group.this.name
}

output "storage_account_name" {
  value = azurerm_storage_account.this.name
}

output "storage_dfs_url" {
  value = azurerm_storage_account.this.primary_dfs_endpoint
}

output "data_factory_name" {
  value = azurerm_data_factory.this.name
}

output "data_factory_managed_identity_object_id" {
  value = azurerm_data_factory.this.identity[0].principal_id
}

output "sql_server_fqdn" {
  value = azurerm_mssql_server.this.fully_qualified_domain_name
```

Terraform source - outputs.tf - Continued

```
}

output "sql_database_name" {
  value = azurerm_mssql_database.this.name
}

output "key_vault_uri" {
  value = azurerm_key_vault.this.vault_uri
}
```

Terraform source - terraform.tfvars.example

```
subscription_id = "<YOUR_SUBSCRIPTION_ID>"
location            = "centralindia"
resource_group_name = "rg-azde-poc01-dev"

storage_account_name = "stazdepoc01CHANGE"
data_factory_name    = "adf-azde-poc01-CHANGE"
sql_server_name      = "sql-azde-poc01-CHANGE"
sql_database_name    = "sqldb-azde-poc01-dev"
key_vault_name       = "kv-azde-poc01-CHANGE"

# NEVER commit real credentials. Copy this file to terraform.tfvars locally.
sql_admin_login    = "pocsqladmin"
sql_admin_password = "CHANGE_ME_LOCALLY"

# Optional: your current public IPv4 address for SQL client access.
client_ip = null

# Optional: your Microsoft Entra object ID so the local Python uploader has blob data access.
uploader_object_id = null

create_log_analytics = false
```

14. Monitoring checklist

- ADF Studio -> Monitor -> Pipeline runs.
- Capture pipeline name, Run ID, status, start/end time and duration.
- Open Copy_Orders_To_Staging and record Rows read and Rows written.
- For failed runs, capture the first failing activity, error code/message and duration.
- Verify SQL counts and ETL log independently.
- Verify landing/archive paths directly in ADLS.

Optional Log Analytics

Create a small Log Analytics workspace only if budget permits. Route selected ADF diagnostics and remove the lab after evidence capture.

Security and GitHub hygiene

- Prefer Managed Identity when the target supports Microsoft Entra authentication.
- Do not commit .env, terraform.tfvars, *.tfstate, Storage keys, SAS tokens, SQL passwords or application client secrets.
- Assign the narrowest role at the narrowest practical scope.
- The beginner lab uses Storage Blob Data Contributor because ADF must read landing, write archive/quarantine and delete processed landing files.
- Key Vault is useful when a real secret is unavoidable, but the main data path does not require one.

Cleanup - stop POC costs

Azure CLI cleanup

```
az group show -n rg-azde-poc01-dev -o table
az resource list -g rg-azde-poc01-dev -o table

# Delete only after evidence capture
az group delete -n rg-azde-poc01-dev --yes
```

- Capture only sanitized architecture, successful run, SQL counts, archive evidence and controlled failures.
- Confirm no unrelated resources are in the Resource Group before deleting.
- After deletion, verify no separately created resource remains outside the POC Resource Group.

15. Interview questions - Part 1

Why land files in ADLS before Azure SQL?

It decouples ingestion from relational processing, preserves raw evidence for audit/replay and lets downstream processing retry without asking the source system to resend.

How is the pipeline idempotent?

The curated table uses order_id as the business key with MERGE update/insert behavior. The advanced repository adds file-level processed control as a second layer.

What is a watermark?

A persisted marker of successful progress, usually a timestamp or source key. The advanced repository updates the watermark only inside a successful transaction.

Managed Identity vs Key Vault secret?

Managed Identity is preferred when the target supports Microsoft Entra authentication because there is no application secret to store or rotate.

Why use staging?

It isolates ingestion from curated logic and makes validation, replay, troubleshooting and transaction-controlled merge easier.

Interview questions - Part 2

When is SHIR required?

When ADF must reach on-premises, local or network-isolated sources that Azure Integration Runtime cannot reach directly.

Why test by removing RBAC?

It proves authorization is real, failures are observable and restoring the correct permission allows safe retry.

How would this scale to thousands of files?

Use metadata-driven control, folder enumeration, ForEach with controlled parallelism, partitioned landing conventions, triggers, retry policies and appropriate compute for heavy transformations.

How would Private Endpoints change the design?

Restrict public endpoints, use ADF Managed Virtual Network and managed private endpoints to ADLS/SQL/Key Vault, while retaining RBAC and SQL database authorization.

Azure RBAC vs SQL permissions?

Azure RBAC controls Azure resource/data-plane access such as ADLS. Azure SQL has a separate database authorization model, so the ADF identity must also exist as a database principal.

POC completion checklist

- [ ] Resource Group created and tagged
- [ ] ADLS Gen2 created with landing/archive/quarantine
- [ ] Azure SQL logical server/database created
- [ ] SQL firewall and Microsoft Entra admin configured
- [ ] ADF created with system-assigned Managed Identity
- [ ] ADF Storage Blob Data Contributor assigned
- [ ] ADF SQL contained user created
- [ ] Linked services test successfully
- [ ] Datasets parameterized

POC completion checklist - Continued

- [ ] PL_INGEST_ORDERS_BATCH validated and published
- [ ] 5-row manual file processed successfully
- [ ] SQL count remains 5 on duplicate rerun
- [ ] Archive and landing-delete behavior verified
- [ ] Missing-file negative test captured
- [ ] RBAC failure/recovery test captured
- [ ] Python upload works through DefaultAzureCredential
- [ ] Terraform workflow understood/tested separately
- [ ] Resource cleanup completed after evidence capture

Beginner replay sequence - one-page revision

Replay sequence

```
1. Create Resource Group and optional budget
2. Create ADLS Gen2 + landing/archive/quarantine
3. Create Azure SQL server + database
4. Configure firewall + Microsoft Entra admin
5. Create ADF and assign Storage Blob Data Contributor
6. Create SQL tables + usp_merge_orders + ADF database user
7. Create ADF linked services
8. Create parameterized datasets
9. Build PL_INGEST_ORDERS_BATCH
10. Validate + Publish
11. Upload 5-row order_date CSV
12. Run ADF
13. Verify dbo.orders count = 5 and ETL log SUCCESS
14. Verify archive and landing delete
15. Re-upload same file and prove count stays 5
16. Run missing-file and RBAC negative tests
17. Install Azure CLI; az login; run Python uploader
18. Re-run ADF and verify Monitor + SQL
19. Recreate separately with Terraform
20. Capture sanitized evidence and clean up
```

Final result

The POC demonstrates a secure, parameterized Azure batch ingestion pattern from ADLS Gen2 through Azure Data Factory into Azure SQL, with idempotent business-key MERGE behavior, raw-file archival, execution logging, Managed Identity/RBAC security, Python-based landing and a Terraform recreation path.

Success criteria to remember

A beginner replay is successful when the source lands in ADLS, ADF completes the intended activities, dbo.orders contains the expected business rows without duplicates, the ETL log shows SUCCESS, the raw file is archived, negative tests fail predictably, and the environment can be cleaned up safely.